Contents

1. Setup and Utilities

Precompile stdc++.h

```
sudo g++ -x c++-header -std=c++17 -O0 stdc++.h -o stdc++.h.gch
```

Pragmas

```
#pragma GCC optimize("Ofast")
#pragma GCC optimize ("unroll-loops")
#pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,avx,tune=native")
```

Random mt19937

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
ll rnd(ll l, ll r) {
    static mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().count());
    return uniform_int_distribution<ll>(l, r)(gen);
}
```

Direction Arrays

```
int dx[8] = { 2, 1, -1, -2, -2, -1, 1, 2 };
int dy[8] = { 1, 2, 2, 1, -1, -2, -2, -1 }; // knight

int dx[8] = {-1,0,1,-1,1,-1,0,1};
int dy[8] = {-1,-1,-1,0,0,1,1,1}; // king

int dx[4] = {1, -1, 0, 0};
int dy[4] = {0, 0, -1, 1};
string direction = "DULR";
```

Custom Comparator

```
struct cmp {
    bool operator() (int a, int b) const {
        return ...;
    }
};
set<int, cmp> s;
```

Fast HashMap (gp_hash_table)

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
struct chash {
    const int RANDOM = (ll)(make_unique<char>().get()) ^
        chrono::high_resolution_clock::now().time_since_epoch().count();

    static unsigned long long hash_f(unsigned long long x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
```

```
    }
    static unsigned hash_combine(unsigned a, unsigned b) {
        return a * 31 + b;
    }
    int operator()(int x) const {
        return hash_f(x)^RANDOM;
    }
};
gp_hash_table<int, int, chash> table;
```

__int128_t

```
istream& operator>>(istream& is, __int128_t& v) {
    string s; is >> s; v = 0;
    for (char c : s) if (isdigit(c)) v = v * 10 + c - '0';
    if (s[0] == '-') v = -v;
    return is;
}
ostream& operator<<(ostream& os, __int128_t v) {
    if (!v) return os << "0";
    if (v < 0) os << '-', v = -v;
    string s;
    while (v) s += '0' + v % 10, v /= 10;
    reverse(s.begin(), s.end());
    return os << s;
}
```

2. Data Structures

2D Prefix Sum

```
// construction
for (int i = 1; i <= N; i++) {
    for (int j = 1; j <= N; j++) {
        prefix[i][j] =
            arr[i][j]
            + prefix[i - 1][j]
            + prefix[i][j - 1]
            - prefix[i - 1][j - 1];
    }
}

// query
pfx[to_row][to_col]
- pfx[from_row - 1][to_col]
- pfx[to_row][from_col - 1]
+ pfx[from_row - 1][from_col - 1];

// partial: add value v to subrectangle [from_row, from_col] to [to_row, to_col]
diff[from_row][from_col]     += v;
diff[from_row][to_col + 1] -= v;
diff[to_row + 1][from_col] -= v;
diff[to_row + 1][to_col + 1] += v;
// then construct diff, grid[i][j] += diff[i][j];
```

BIT, Fenwick Tree

```
struct BIT { // 0-based
    int n;
    vector<ll> tree;

    BIT(int size) : n(size + 2), tree(n + 1) {}
    void add(int i, ll val) {
        for(i++; i <= n; i += i & -i) tree[i] += val;
    }
    ll query(int i) {
        ll sum = 0;
        for (i++; i > 0; i &= i - 1) sum += tree[i];
        return sum;
    }
    int lower_bound(ll target) {
        int i = 0;
        ll curr = 0;
        for (int mask = 1 << __lg(n); mask > 0; mask >>= 1) {
            if (i + mask <= n && curr + tree[i + mask] < target) {
                curr += tree[i += mask];
            }
        }
        return i;
    }
};
```

BIT Range / Fenwick Range

```
template<typename T>
class BitR { // 0-based
    int n;
    vector<T> f, s;
    void add(vector<T> &a, int i, T val) {
        for(; i < n; i += i & -i)
            a[i] += val;
    }
public:
    BitR(int n) : n(n + 5), f(n + 6), s(n + 6) { }

    void add(int i, T val) { add(s, i + 1, -val); }

    T query_point(int i) {
        if (!i) return query(0);
        return query(i) - query(i - 1);
    }
    void set(int i, T val) {
        int c = query_point(i);
        add(i, val - c);
    }

    void add(int l, int r, T val) {
        l++, r++;
        add(f, l, val);
        add(f, r + 1, -val);
        add(s, l, val * (l - 1));
        add(s, r + 1, -val * r);
    }

    T query(int ii) {
        ii++;
        T sum = 0;
        int i = ii;
        for(; i > 0; i ^= i & -i)
            sum += f[i];
        sum *= ii;
        i = ii;
        for(; i > 0; i ^= i & -i)
            sum -= s[i];
        return sum;
    }

    T range(int l, int r) { return query(r) - query(l - 1); }
};
```

2D BIT / 2D fenwick

```
template<typename T>
struct BIT2D { // 1-based
    int n, m;
    vector<vector<T>> tree;

    BIT2D(int n, int m) : n(n), m(m), tree(n + 2, vector<T>(m + 2, 0)) {}

    void update(int x, int y, T val) {
```

```
        for (int i = x; i <= n; i += i & -i) {
            for (int j = y; j <= m; j += j & -j) {
                tree[i][j] += val;
            }
        }
    }

    T getPrefix(int x, int y) {
        if (x <= 0 || y <= 0) return 0;
        T ret = 0; // change default value
        for (int i = x; i > 0; i &= i - 1) {
            for (int j = y; j > 0; j &= j - 1) {
                ret += tree[i][j];
            }
        }
        return ret;
    }

    T getSquare(int xl, int yl, int xr, int yr) { // change operation
        return getPrefix(xr, yr) + getPrefix(xl - 1, yl - 1) -
               getPrefix(xr, yl - 1) - getPrefix(xl - 1, yr);
    }
};
```

Segment Tree

```
struct info {
    long long sum = 0;
    info(long long sum = 0) : sum(sum) {}
    friend info operator+(const info &l, const info &r) {
        return {l.sum + r.sum};
    }
};

template<class info>
struct segmentTreeIterative {
    int n;
    vector<info> tree;

    explicit segmentTreeIterative(int n) : n(n), tree(n << 1) {}

    template<class U>
    explicit segmentTreeIterative(const vector<U> &arr) : n(arr.size()),
                tree(n << 1) {
        for(int i = 0; i < n; i++) tree[i + n] = info(arr[i]);
        for(int i = n - 1; i > 0; i--) tree[i] = tree[i << 1] + tree[i << 1 | 1];
    }

    void set(int i, info v) {
        for(tree[i += n] = v; i >>= 1; )
            tree[i] = tree[i << 1] + tree[i << 1 | 1];
    }

    info get(int l, int r) {
        info resL, resR;
        for(l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if(l & 1) resL = resL + tree[l++];
```

```
            if(r & 1) resR = tree[--r] + resR;
        }
        return resL + resR;
    }
};
```

Segment Tree (Recursive)

```
struct info {
    long long sum = 0;
    info(long long sum = 0) : sum(sum) {}
    friend info operator+(const info &l, const info &r) {
        return {l.sum + r.sum};
    }
};

template<class info>
struct dynamicSegmentTree {
    struct node { int l = 0, r = 0; info v; };
    vector<node> tr;
    int n, root = 0;

    explicit dynamicSegmentTree(int n = 1e9, int expectedOps = 1) : n(n), tr(1) {
        tr.reserve(expectedOps * __lg(n) * 2);
    }

    info get(int x, int lx, int rx, int l, int r) {
        if (!x || lx > r || l > rx) return info();
        if (lx >= l && rx <= r) return tr[x].v;
        int m = (lx + rx) >> 1;
        return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
    }

    int set(int x, int lx, int rx, int i, info val) {
        if (i < lx || i > rx) return x;
        if (!x) x = tr.size(), tr.emplace_back();
        if (lx == rx) return tr[x].v = val, x;
        int m = (lx + rx) >> 1;
        tr[x].l = set(tr[x].l, lx, m, i, val);
        tr[x].r = set(tr[x].r, m + 1, rx, i, val);
        return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
    }

    void set(int i, info v) { root = set(root, 0, n, i, v); }
    info get(int l, int r) { return get(root, 0, n, l, r); }
};
```

2D Segment Tree

```
struct info {
    long long sum = 0;
    info(long long sum = 0) : sum(sum) {}
    friend info operator+(const info &l, const info &r) {
        return {l.sum + r.sum};
    }
};
```

```
template<class info>
struct segmentTree2d {
    int nx, ny;
    vector<vector<info>> tree;

    explicit segmentTree2d(int n, int m) : nx(n), ny(m), tree(nx << 1,
        vector<info>(ny << 1)) {}

    template<class U>
    explicit segmentTree2d(const vector<vector<U>> &a) {
        nx = a.size();
        ny = nx ? a[0].size() : 0;
        tree.assign(nx << 1, vector<info>(ny << 1));

        for(int i = 0; i < nx; i++) {
            for(int j = 0; j < ny; j++) tree[i + nx][j + ny] = info(a[i][j]);
            for(int y = ny - 1; y > 0; y--)
                tree[i + nx][y] = tree[i + nx][y << 1] + tree[i + nx][y << 1 | 1];
        }
        for(int x = nx - 1; x > 0; x--)
            for(int y = 0; y < (ny << 1); y++)
                tree[x][y] = tree[x << 1][y] + tree[x << 1 | 1][y];
    }

    void set(int i, int j, info v) {
        if (i < 0 || i >= nx || j < 0 || j >= ny) return;
        int x = i + nx, y = j + ny;
        tree[x][y] = v;
        for(int yy = y >> 1; yy > 0; yy >>= 1)
            tree[x][yy] = tree[x][yy << 1] + tree[x][yy << 1 | 1];

        for(int xx = x >> 1; xx > 0; xx >>= 1) {
            tree[xx][y] = tree[xx << 1][y] + tree[xx << 1 | 1][y];
            for(int yy = y >> 1; yy > 0; yy >>= 1)
                tree[xx][yy] = tree[xx][yy << 1] + tree[xx][yy << 1 | 1];
        }
    }

    info queryY(int nodeX, int y1, int y2) const {
        info resL, resR;
        for(int l = y1 + ny, r = y2 + ny + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) resL = resL + tree[nodeX][l++];
            if (r & 1) resR = tree[nodeX][--r] + resR;
        }
        return resL + resR;
    }

    info get(int x1, int y1, int x2, int y2) {
        if (!nx || !ny) return info();
        x1 = max(x1, 0); y1 = max(y1, 0);
        x2 = min(x2, nx - 1); y2 = min(y2, ny - 1);
        if (x1 > x2 || y1 > y2) return info();

        info resL, resR;
        for(int l = x1 + nx, r = x2 + nx + 1; l < r; l >>= 1, r >>= 1) {
```

```
            if (l & 1) resL = resL + queryY(l++, y1, y2);
            if (r & 1) resR = queryY(--r, y1, y2) + resR;
        }
        return resL + resR;
    }
};
```

Lazy Segment Tree

```
struct tag {
    ll add = 0;
    void apply(const tag &p) {
        add += p.add;
    }
};

struct info {
    ll sum = 0;
    info(ll sum = 0) : sum(sum) {}
    void apply(const tag &lz, int lx, int rx) {
        sum += 1ll * (rx - lx + 1) * lz.add;
    }
    friend info operator+(const info &l, const info &r) {
        return {l.sum + r.sum};
    }
};

template<class info, class tag>
struct lazySegment {
    int n;
    vector<info> tr;
    vector<tag> lz;

    explicit lazySegment(int _n) : n(_n + 1), tr(4 << __lg(n)), lz(4 << __lg(n)) {}
    template<class U>
    explicit lazySegment(const U &arr) : n(arr.size()), tr(4 << __lg(n)),
                lz(4 << __lg(n)) {
        build(1, 0, n - 1, arr);
    }

    void push(int x, int lx, int rx) {
        if (lx != rx) lz[x << 1].apply(lz[x]), lz[x << 1 | 1].apply(lz[x]);
        tr[x].apply(lz[x], lx, rx);
        lz[x] = tag();
    }

    template<class U>
    void build(int x, int lx, int rx, const U &arr) {
        if (lx == rx) return void(tr[x] = arr[lx]);
        int m = (lx + rx) >> 1;
        build(x << 1, lx, m, arr), build(x << 1 | 1, m + 1, rx, arr);
        tr[x] = tr[x << 1] + tr[x << 1 | 1];
    }

    info get(int x, int lx, int rx, int l, int r) {
        push(x, lx, rx);
```

```
        if (lx > r || l > rx) return info();
        if (lx >= l && rx <= r) return tr[x];
        int m = (lx + rx) >> 1;
        return get(x << 1, lx, m, l, r) + get(x << 1 | 1, m + 1, rx, l, r);
    }

    void set(int x, int lx, int rx, int i, info val) {
        push(x, lx, rx);
        if (i < lx || i > rx) return;
        if (lx == rx) return void(tr[x] = val);
        int m = (lx + rx) >> 1;
        set(x << 1, lx, m, i, val), set(x << 1 | 1, m + 1, rx, i, val);
        tr[x] = tr[x << 1] + tr[x << 1 | 1];
    }

    void applyRange(int x, int lx, int rx, int l, int r, tag v) {
        push(x, lx, rx);
        if (lx > r || l > rx) return;
        if (lx >= l && rx <= r) return lz[x] = v, push(x, lx, rx);
        int m = (lx + rx) >> 1;
        applyRange(x << 1, lx, m, l, r, v),
        applyRange(x << 1 | 1, m + 1, rx, l, r, v);
        tr[x] = tr[x << 1] + tr[x << 1 | 1];
    }

    void set(int i, info v) { set(1, 0, n - 1, i, v); }
    void applyRange(int l, int r, tag v) { applyRange(1, 0, n - 1, l, r, v); }
    info get(int l, int r) { return get(1, 0, n - 1, l, r); }
};
```

Dynamic Lazy Segment Tree

```
template<class info, class tag>
struct dynamicLazySegmentTree {
    struct node { int l = 0, r = 0; info v; tag t; };
    vector<node> tr;
    int n, root = 0;

    explicit dynamicLazySegmentTree(int n = 1e9) : n(n), tr(1) {}

    void push(int x, int lx, int rx) {
        if (lx == rx) return void(tr[x].t = tag());
        if (!tr[x].l) tr[x].l = tr.size(), tr.emplace_back();
        if (!tr[x].r) tr[x].r = tr.size(), tr.emplace_back();

        int m = (lx + rx) >> 1, l = tr[x].l, r = tr[x].r;
        tr[l].t.apply(tr[x].t), tr[l].v.apply(tr[x].t, lx, m);
        tr[r].t.apply(tr[x].t), tr[r].v.apply(tr[x].t, m + 1, rx);
        tr[x].t = tag();
    }

    int set(int x, int lx, int rx, int i, info val) {
        if (i < lx || i > rx) return x;
        if (!x) x = tr.size(), tr.emplace_back();
        if (lx == rx) {
```

```
            tr[x].v = val, tr[x].t = tag();
            return x;
        }
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        tr[x].l = set(tr[x].l, lx, m, i, val);
        tr[x].r = set(tr[x].r, m + 1, rx, i, val);
        return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
    }

    int applyRange(int x, int lx, int rx, int l, int r, tag val) {
        if (lx > r || l > rx) return x;
        if (!x) x = tr.size(), tr.emplace_back();
        if (lx >= l && rx <= r) {
            tr[x].t.apply(val);
            tr[x].v.apply(val, lx, rx);
            return x;
        }
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        tr[x].l = applyRange(tr[x].l, lx, m, l, r, val);
        tr[x].r = applyRange(tr[x].r, m + 1, rx, l, r, val);
        return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
    }

    info get(int x, int lx, int rx, int l, int r) {
        if (!x || lx > r || l > rx) return info();
        if (lx >= l && rx <= r) return tr[x].v;
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
    }

    void set(int i, info v) { root = set(root, 0, n, i, v); }
    void applyRange(int l, int r, tag v) { root = applyRange(root, 0, n, l, r, v); }
    info get(int l, int r) { return get(root, 0, n, l, r); }
};
```

Segment Tree Beats

```
#include <numeric> // Required for gcd

struct segtreebeats {
    static const ll INF = 2e18, UNSET = LLONG_MIN;

    struct Node {
        ll sum, mx1, mx2, mxc, mn1, mn2, mnc, d_gcd, lz_add, lz_set;
    };

    int sz;
    vector<Node> tree;

    Node leaf(ll v) { return {v, v, -INF, 1, v, INF, 1, 0, 0, UNSET}; }

    segtreebeats(int n, const vector<ll> &a) {
        for (sz = 1; sz < n; sz <<= 1);
```

```
        tree.assign(sz << 1, leaf(0));
        build(a, n, 0, 0, sz - 1);
    }

    Node merge(const Node &L, const Node &R) {
        Node U = {L.sum + R.sum, 0, 0, 0, 0, 0, 0, gcd(L.d_gcd, R.d_gcd), 0, UNSET};

if (L.mx1 == R.mx1) U.mx1 = L.mx1, U.mx2 = max(L.mx2, R.mx2), U.mxc = L.mxc + R.mxc;
else if (L.mx1 > R.mx1) U.mx1 = L.mx1, U.mx2 = max(L.mx2, R.mx1), U.mxc = L.mxc;
else U.mx1 = R.mx1, U.mx2 = max(L.mx1, R.mx2), U.mxc = R.mxc;

if (L.mn1 == R.mn1) U.mn1 = L.mn1, U.mn2 = min(L.mn2, R.mn2), U.mnc = L.mnc + R.mnc;
else if (L.mn1 < R.mn1) U.mn1 = L.mn1, U.mn2 = min(L.mn2, R.mn1), U.mnc = L.mnc;
else U.mn1 = R.mn1, U.mn2 = min(L.mn1, R.mn2), U.mnc = R.mnc;

        ll aL = L.mx2, aR = R.mx2;
        if (aL != -INF && aL != L.mn1 && aR != -INF && aR != R.mn1)
            U.d_gcd = gcd(U.d_gcd, abs(aL - aR));

        ll any = UNSET;
        if (aL != -INF && aL != L.mn1) any = aL;
        else if (aR != -INF && aR != R.mn1) any = aR;

        for (ll val : {L.mn1, L.mx1, R.mn1, R.mx1}) {
            if (val != U.mn1 && val != U.mx1) {
                if (any != UNSET) U.d_gcd = gcd(U.d_gcd, abs(val - any));
                else any = val;
            }
        }
        return U;
    }

    void apply_set(int x, int lx, int rx, ll v) {
        ll len = rx - lx + 1;
        tree[x] = {len * v, v, -INF, len, v, INF, len, 0, 0, v};
    }

    void apply_add(int x, int lx, int rx, ll v) {
        if (!v) return;
        auto &nd = tree[x];
        if (nd.lz_set != UNSET) return apply_set(x, lx, rx, nd.lz_set + v);
        if (nd.mx1 == nd.mn1) return apply_set(x, lx, rx, nd.mn1 + v);
        ll len = rx - lx + 1;
        nd.sum += len * v;
        nd.mx1 += v; if (nd.mx2 != -INF) nd.mx2 += v;
        nd.mn1 += v; if (nd.mn2 != INF) nd.mn2 += v;
        nd.lz_add += v;
    }

    void apply_chmin(int x, int lx, int rx, ll v) {
        auto &nd = tree[x];
        if (nd.mx1 <= v) return;
        if (nd.mn1 >= v) return apply_set(x, lx, rx, v);
        if (nd.mn2 == nd.mx1) nd.mn2 = v;
        nd.sum -= (nd.mx1 - v) * nd.mxc;
        nd.mx1 = v;
```

```
    }

    void apply_chmax(int x, int lx, int rx, ll v) {
        auto &nd = tree[x];
        if (nd.mn1 >= v) return;
        if (nd.mx1 <= v) return apply_set(x, lx, rx, v);
        if (nd.mx2 == nd.mn1) nd.mx2 = v;
        nd.sum += (v - nd.mn1) * nd.mnc;
        nd.mn1 = v;
    }

    void push(int x, int lx, int rx) {
        if (lx == rx) return void(tree[x].lz_add = 0, tree[x].lz_set = UNSET);
        int m = (lx + rx) >> 1, lc = x * 2 + 1, rc = x * 2 + 2;

        if (tree[x].lz_set != UNSET) {
            apply_set(lc, lx, m, tree[x].lz_set);
            apply_set(rc, m + 1, rx, tree[x].lz_set);
            tree[x].lz_set = UNSET;
        }
        if (tree[x].lz_add) {
            apply_add(lc, lx, m, tree[x].lz_add);
            apply_add(rc, m + 1, rx, tree[x].lz_add);
            tree[x].lz_add = 0;
        }
        if (tree[lc].mx1 > tree[x].mx1) apply_chmin(lc, lx, m, tree[x].mx1);
        if (tree[rc].mx1 > tree[x].mx1) apply_chmin(rc, m + 1, rx, tree[x].mx1);
        if (tree[lc].mn1 < tree[x].mn1) apply_chmax(lc, lx, m, tree[x].mn1);
        if (tree[rc].mn1 < tree[x].mn1) apply_chmax(rc, m + 1, rx, tree[x].mn1);
    }

    void build(const vector<ll> &a, int n, int x, int lx, int rx) {
        if (lx == rx) return void(tree[x] = (lx < n) ? leaf(a[lx]) : leaf(0));
        int m = (lx + rx) >> 1;
        build(a, n, x * 2 + 1, lx, m);
        build(a, n, x * 2 + 2, m + 1, rx);
        tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
    }

    void chmin(int l, int r, ll v, int x, int lx, int rx) {
        if (lx > r || rx < l || tree[x].mx1 <= v) return;
        if (lx >= l && rx <= r && tree[x].mx2 < v) return apply_chmin(x, lx, rx, v);
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        chmin(l, r, v, x * 2 + 1, lx, m);
        chmin(l, r, v, x * 2 + 2, m + 1, rx);
        tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
    }

    void chmax(int l, int r, ll v, int x, int lx, int rx) {
        if (lx > r || rx < l || tree[x].mn1 >= v) return;
        if (lx >= l && rx <= r && tree[x].mn2 > v) return apply_chmax(x, lx, rx, v);
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        chmax(l, r, v, x * 2 + 1, lx, m);
```

```
        chmax(l, r, v, x * 2 + 2, m + 1, rx);
        tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
    }

    void assign(int l, int r, ll v, int x, int lx, int rx) {
        if (lx > r || rx < l) return;
        if (lx >= l && rx <= r) return apply_set(x, lx, rx, v);
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        assign(l, r, v, x * 2 + 1, lx, m);
        assign(l, r, v, x * 2 + 2, m + 1, rx);
        tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
    }

    void add(int l, int r, ll v, int x, int lx, int rx) {
        if (lx > r || rx < l) return;
        if (lx >= l && rx <= r) return apply_add(x, lx, rx, v);
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        add(l, r, v, x * 2 + 1, lx, m);
        add(l, r, v, x * 2 + 2, m + 1, rx);
        tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
    }

    ll sum(int l, int r, int x, int lx, int rx) {
        if (lx > r || rx < l) return 0;
        if (lx >= l && rx <= r) return tree[x].sum;
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        return sum(l, r, x * 2 + 1, lx, m) + sum(l, r, x * 2 + 2, m + 1, rx);
    }

    ll qmin(int l, int r, int x, int lx, int rx) {
        if (lx > r || rx < l) return INF;
        if (lx >= l && rx <= r) return tree[x].mn1;
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        return min(qmin(l, r, x * 2 + 1, lx, m), qmin(l, r, x * 2 + 2, m + 1, rx));
    }

    ll qmax(int l, int r, int x, int lx, int rx) {
        if (lx > r || rx < l) return -INF;
        if (lx >= l && rx <= r) return tree[x].mx1;
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        return max(qmax(l, r, x * 2 + 1, lx, m), qmax(l, r, x * 2 + 2, m + 1, rx));
    }

    ll qgcd(int l, int r, int x, int lx, int rx) {
        if (lx > r || rx < l) return 0;
        if (lx >= l && rx <= r) {
            ll ans = gcd(tree[x].d_gcd, abs(tree[x].mx1));
            if (tree[x].mx2 != -INF) ans = gcd(ans, abs(tree[x].mx2 - tree[x].mx1));
            if (tree[x].mn2 != INF) ans = gcd(ans, abs(tree[x].mn2 - tree[x].mn1));
            return ans;
```

```
        }
        push(x, lx, rx);
        int m = (lx + rx) >> 1;
        return gcd(qgcd(l, r, x * 2 + 1, lx, m), qgcd(l, r, x * 2 + 2, m + 1, rx));
    }

    // Public Wrappers
    void chmin(int l, int r, ll v) { chmin(l, r, v, 0, 0, sz - 1); }
    void chmax(int l, int r, ll v) { chmax(l, r, v, 0, 0, sz - 1); }
    void assign(int l, int r, ll v) { assign(l, r, v, 0, 0, sz - 1); }
    void add(int l, int r, ll v) { add(l, r, v, 0, 0, sz - 1); }
    ll sum(int l, int r) { return sum(l, r, 0, 0, sz - 1); }
    ll qmin(int l, int r) { return qmin(l, r, 0, 0, sz - 1); }
    ll qmax(int l, int r) { return qmax(l, r, 0, 0, sz - 1); }
    ll qgcd(int l, int r) { return qgcd(l, r, 0, 0, sz - 1); }
};
```

Dynamic Persistent Segment Tree

```
template<class info>
struct DPSGT {
    struct node { int l = 0, r = 0; info v; };
    vector<node> tr;
    int n;

    explicit DPSGT(int n = 1e9, int expectedOps = 1) : n(n), tr(1) {
        tr.reserve(expectedOps * __lg(n) * 2);
    }

    info get(int x, int lx, int rx, int l, int r) {
        if (!x || lx > r || l > rx) return info();
        if (lx >= l && rx <= r) return tr[x].v;
        int m = (lx + rx) >> 1;
        return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
    }

    int set(int x, int lx, int rx, int i, info val) {
        if (i < lx || i > rx) return x;
        int nx = tr.size();
        tr.push_back(tr[x]);
        if (lx == rx) return tr[nx].v = val, nx;
        int m = (lx + rx) >> 1;
        tr[nx].l = set(tr[nx].l, lx, m, i, val);
        tr[nx].r = set(tr[nx].r, m + 1, rx, i, val);
        return tr[nx].v = tr[tr[nx].l].v + tr[tr[nx].r].v, nx;
    }

    int set(int root, int i, info v) { return set(root, 0, n, i, v); }
    info get(int root, int l, int r) { return get(root, 0, n, l, r); }
};
```

Wavelet Tree

```
struct WaveletTree {
    int lo, hi;
```

```
    WaveletTree *lc = 0, *rc = 0;
    vector<int> b;

    template<class It>
    WaveletTree(It from, It to, int x, int y) : lo(x), hi(y) {
        if (from >= to || lo == hi) return;
        int mid = lo + (hi - lo) / 2;

        b.reserve(distance(from, to) + 1);
        b.push_back(0);
        for (auto it = from; it != to; ++it) {
            b.push_back(b.back() + (*it <= mid));
        }

        auto pivot = stable_partition(from, to, [mid](int val)
            { return val <= mid; });
        lc = new WaveletTree(from, pivot, lo, mid);
        rc = new WaveletTree(pivot, to, mid + 1, hi);
    }

    // the passed vector is changed, pass a copy in the main
    // Takes a vector of integers, and the min/max values in it.
    // The vector should be 0-indexed but queries works as 1-based
    WaveletTree(vector<int>& a, int x, int y) :
    WaveletTree(a.begin(), a.end(), x, y) {}

    ~WaveletTree() { delete lc; delete rc; }

    int kth(int l, int r, int k) {
        if (l > r) return 0;
        if (lo == hi) return lo;
        int in_left = b[r] - b[l - 1], lb = b[l - 1], rb = b[r];
        if (k <= in_left) return lc->kth(lb + 1, rb, k);
        return rc->kth(l - lb, r - rb, k - in_left);
    }

    int LTE(int l, int r, int k) {
        if (l > r || k < lo) return 0;
        if (hi <= k) return r - l + 1;
        int lb = b[l - 1], rb = b[r];
        return lc->LTE(lb + 1, rb, k) + rc->LTE(l - lb, r - rb, k);
    }

    int count(int l, int r, int k) {
        if (l > r || k < lo || k > hi) return 0;
        if (lo == hi) return r - l + 1;
        int mid = lo + (hi - lo) / 2, lb = b[l - 1], rb = b[r];
        if (k <= mid) return lc->count(lb + 1, rb, k);
        return rc->count(l - lb, r - rb, k);
    }
};
```

Implicit Treap

```
// You must manually track the root of your tree in your main function:
//      Treap tree;
```

```
//      int root = 0;
// 1. Insert val at index i:
//      int l, r;
//      tree.split(root, i, l, r);
//      int mid = tree.new_node(val);
//      root = tree.merge(tree.merge(l, mid), r);
//
// 2. Delete the element at index i:
//      auto [l, mid, r] = tree.split(root, i, i);
//      root = tree.merge(l, r); // mid is safely dropped and ignored
//
// 3. Range Sum for subarray [L, R]:
//      auto [l, mid, r] = tree.split(root, L, R);
//      long long ans = tree.tr[mid].sum;
//      root = tree.merge(tree.merge(l, mid), r); // ALWAYS merge back!
//
// 4. Range Reverse for subarray [L, R]:
//      auto [l, mid, r] = tree.split(root, L, R);
//      tree.tr[mid].rev ^= 1; // apply the lazy tag
//      root = tree.merge(tree.merge(l, mid), r); // ALWAYS merge back!
// ------------------------------------------

mt19937 rnd(time(nullptr));
struct Treap {
    struct node {
        uint32_t pri = rnd();
        int sz = 1, l = 0, r = 0, val{};
        int64_t sum{};
        bool rev = false;
        node(int x) : sum(val = x) { }
    };
    vector<node> tr;

    // tr[0] acts as a dummy/null node to avoid segmentation faults
    Treap() : tr(1, 0) { tr[0].sz = 0; }

    inline void pull(int x) {
        tr[x].sz = tr[tr[x].l].sz + tr[tr[x].r].sz + 1;
        tr[x].sum = tr[tr[x].l].sum + tr[tr[x].r].sum + tr[x].val;
    }

    inline void push(int x) {
        if(tr[x].rev) {
            tr[tr[x].l].rev ^= 1, tr[tr[x].r].rev ^= 1;
            swap(tr[x].l, tr[x].r);
            tr[x].rev = false;
        }
    }

    // Returns the index of the newly created node
    inline int new_node(int val) {
        tr.emplace_back(val);
        return int(tr.size()) - 1;
    }

    // Concatenates treap rx to the right of lx. Returns the new root.
    int merge(int lx, int rx) {
```

```
        if(!lx || !rx) return rx + lx;
        push(lx), push(rx);
        if(tr[lx].pri < tr[rx].pri)
            return tr[rx].l = merge(lx, tr[rx].l), pull(rx), rx;
        return tr[lx].r = merge(tr[lx].r, rx), pull(lx), lx;
    }

    // Splits treap x into lx (first k elements) and rx (the rest).
    void split(int x, int k, int &lx, int &rx) {
        if(!x) return lx = rx = 0, void();
        push(x);
        if(k <= tr[tr[x].l].sz) split(tr[x].l, k, lx, tr[x].l), pull(rx = x);
        else split(tr[x].r, k - tr[tr[x].l].sz - 1, tr[x].r, rx), pull(lx = x);
    }

    // Helper to extract a specific subarray [l, r] into the middle partition
    array<int, 3> split(int x, int l, int r) {
        int a, b, c;
        split(x, r + 1, b, c);
        split(b, l, a, b);
        return {a, b, c};
    }
};
```

Sparse Table

```
struct ST {
    static const int K = 18, N = 2e5 + 5;
    int st[K][N];

    void build(int n, const vector<int>& a) {
        for(int i = 0; i < n; i++) st[0][i] = a[i];
        for(int k = 1; k < K; k++)
            for(int i = 0; i + (1 << k) <= n; i++)
                st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }

    int query(int l, int r) {
        int k = __lg(r - l + 1);
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
};


template<class T, class F>
struct sparse {
    int n, Log;
    vector<vector<T>> table;
    F merge;
    T id;

    sparse(const vector<T>& arr, F merge, T id = T()) :
            n(arr.size()), Log(__lg(n) + 1), table(Log, vector<T>(n)),
            merge(merge), id(id) {
        table[0] = arr;
        for (int l = 1; l < Log; l++) {
```

```
            for (int i = 0; i + (1 << (l - 1)) < n; i++) {
                table[l][i] = merge(table[l - 1][i],
                table[l - 1][i + (1 << (l - 1))]);
            }
        }
    }

    T query(int l, int r) {
        if (l > r) return id;
        int len = __lg(r - l + 1);
        return merge(table[len][l], table[len][r - (1 << len) + 1]);
    }

    T query_log(int l, int r) {
        T res = id;
        bool first = true;
        for (int j = Log - 1; j >= 0; j--) {
            if (1 << j <= r - l + 1) {
                res = first ? table[j][l] : merge(res, table[j][l]);
                first = false;
                l += 1 << j;
            }
        }
        return res;
    }
};
// sparse st(a, [](int x, int y) { return min(x, y); }, INT_MAX);
```

2D Sparse Table

```
#include <bits/stdc++.h>
using namespace std;

template<class T, class F>
struct sparse2d {
    int n, m, Kx, Ky;
    vector<int> lgx, lgy;
    vector<vector<vector<vector<T>>>> st;
    F merge;

    sparse2d(const vector<vector<T>>& a, F merge) : merge(merge) {
        n = (int)a.size();
        m = (int)a[0].size();

        lgx.resize(n + 1);
        lgy.resize(m + 1);
        for (int i = 2; i <= n; ++i) lgx[i] = lgx[i / 2] + 1;
        for (int j = 2; j <= m; ++j) lgy[j] = lgy[j / 2] + 1;

        Kx = lgx[n];
        Ky = lgy[m];

        st.assign(Kx + 1, vector<vector<vector<T>>>(Ky + 1));

        st[0][0] = a;

        // Build along columns for kx = 0
```

```
        for (int ky = 1; ky <= Ky; ++ky) {
            int len = 1 << ky;
            int half = len >> 1;
            st[0][ky].assign(n, vector<T>(m - len + 1));
            for (int i = 0; i < n; ++i) {
                for (int j = 0; j + len <= m; ++j) {
                    st[0][ky][i][j] = merge(st[0][ky - 1][i][j],
                                            st[0][ky - 1][i][j + half]);
                }
            }
        }

        // Build along rows for all ky
        for (int kx = 1; kx <= Kx; ++kx) {
            int lenx = 1 << kx;
            int halfx = lenx >> 1;
            for (int ky = 0; ky <= Ky; ++ky) {
                int leny = 1 << ky;
                st[kx][ky].assign(n - lenx + 1, vector<T>(m - leny + 1));
                for (int i = 0; i + lenx <= n; ++i) {
                    for (int j = 0; j + leny <= m; ++j) {
                        st[kx][ky][i][j] = merge(st[kx - 1][ky][i][j],
                                                 st[kx - 1][ky][i + halfx][j]);
                    }
                }
            }
        }
    }

    // inclusive coordinates: [x1..x2], [y1..y2]
    T query(int x1, int y1, int x2, int y2) const {
        int kx = lgx[x2 - x1 + 1];
        int ky = lgy[y2 - y1 + 1];
        int dx = 1 << kx;
        int dy = 1 << ky;

        const T& a = st[kx][ky][x1][y1];
        const T& b = st[kx][ky][x2 - dx + 1][y1];
        const T& c = st[kx][ky][x1][y2 - dy + 1];
        const T& d = st[kx][ky][x2 - dx + 1][y2 - dy + 1];

        return merge(merge(a, b), merge(c, d));
    }
};
```

DSU

```
struct DSU {
    vector<int> p, sz;
    int n, comps;
    DSU(int _n = 0) { init(_n); }
    void init(int _n) {
        n = _n + 10; comps = _n;
        p.resize(n); sz.assign(n, 1);
        iota(p.begin(), p.end(), 0);
    }
    int find(int u) { return u == p[u] ? u : p[u] = find(p[u]); }
```

```
    bool unite(int u, int v) {
        u = find(u), v = find(v);
        if (u == v) return 0;
        if (sz[u] < sz[v]) swap(u, v);
        p[v] = u; sz[u] += sz[v]; comps--;
        return 1;
    }
    bool same(int u, int v) { return find(u) == find(v); }
    int size(int u) { return sz[find(u)]; }
    int size() { return comps; }
};
```

Persistent DSU

```
struct persistent_DSU {
    vector<vector<pair<int, int> > > par;
    persistent_DSU(int n) : par(n + 1, {{-1, 0}}) { }

    int time = 0;
    bool merge(int u, int v) {
        ++time;
        if ((u = root(u, time)) == (v = root(v, time)))
            return 0;
        if (par[u].back().first > par[v].back().first)
            swap(u, v);
        par[u].push_back({par[u].back().first + par[v].back().first, time});
        par[v].push_back({u, time}); // par[v] = u
        return 1;
    }

    bool same(int u, int v, int t) {
        return root(u, t) == root(v, t);
    }

    int root(int u, int t) {
        // root of u at time t
        if (par[u].back().first >= 0 && par[u].back().second <= t)
            return root(par[u].back().first, t);
        return u;
    }

    int size(int u, int t) { // O(log)
        // size of the component of u at time t
        u = root(u, t);
        int l = 0, r = (int) par[u].size() - 1, ans = 0;
        while (l <= r) {
            int mid = l + r >> 1;
            if (par[u][mid].second <= t)
                ans = mid, l = mid + 1;
            else
                r = mid - 1;
        }
        return -par[u][ans].first;
    }
};
```

Bipartite DSU

```
// Maintains whether each component is bipartite
struct BipartiteDSU {
    vector<int> sz, bipartite;
    vector<pair<int, int> > par;

    BipartiteDSU(int n) : par(n), sz(n, 1), bipartite(n) {
        for (int i = 0; i < n; ++i) {
            par[i] = {i, 0};
        }
    }

    pair<int, int> find(int u) {
        if (u == par[u].first) return {u, 0};
        int parity = par[u].second;
        par[u] = find(par[u].first);
        par[u].second ^= parity;
        return par[u];
    }

    bool same(int x, int y) {
        return find(x).first == find(y).first;
    }

    bool join(int u, int v) {
        pair<int, int> pu = find(u);
        pair<int, int> pv = find(v);
        u = pu.first;
        v = pv.first;
        int x = pu.second, y = pv.second;
        if (u == v) {
            if (x == y) bipartite[u] = false;
            return false;
        }
        if (sz[u] < sz[v]) swap(u, v);
        par[v] = {u, x ^ y ^ 1};
        bipartite[u] &= bipartite[v];
        sz[u] += sz[v];
        return true;
    }

    int size(int x) { return sz[find(x).first]; }
};
```

Monotonic Stack / Queue

```
template<class T>
struct Mono_stack {
    stack<pair<T, T> > st;

    void push(const T &val) {
        if (st.empty()) st.emplace(val, val);
        else st.emplace(val, max(val, st.top().second));
    }

    void pop() { st.pop(); }
```

```cpp
    bool empty() { return st.empty(); }
    int size() { return st.size(); }
    T top() { return st.top().first; }
    T max() { return st.top().second; }
};

template<class T>
struct Mono_queue {
    Mono_stack<T> pop_st, push_st;

    void push(const T &val) { push_st.push(val); }

    void move() {
        if (pop_st.size()) return;
        while (!push_st.empty()) pop_st.push(push_st.top()), push_st.pop();
    }

    void pop() { move(); pop_st.pop(); }
    bool empty() { return pop_st.empty() && push_st.empty(); }
    int size() { return pop_st.size() + push_st.size(); }
    T top() { move(); return pop_st.top(); }

    T max() {
        if (pop_st.empty()) return push_st.max();
        if (push_st.empty()) return pop_st.max();
        return max(push_st.max(), pop_st.max());
    }
};
```

Ordered Data Structures (pb_ds)

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T> using ordered_set = tree<T, null_type, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
template <typename T, typename R> using ordered_map = tree<T, R, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
```

BucketList

```cpp
template<class T>
struct BucketList {
    int siz = 0;
    vector<vector<T> > a;
    static constexpr int SPLIT_RATIO = 28;

    void insert(int i, T x) {
        if (siz == 0) {
            siz = 1;
            a = {{x}};
            return;
        }
        siz++;
        for (ll bi = 0; bi < size(a); bi++) {
            auto &bucket = a[bi];
```

```
            if (i <= size(bucket)) {
                bucket.insert(begin(bucket) + i, x);
                if (size(bucket) > size(a) * SPLIT_RATIO) {
                    auto L = end(bucket) - size(bucket) / 2, R = end(bucket);
                    a.emplace(begin(a) + bi + 1, L, R);
                    // bucket might be broken
                    a[bi].erase(L, R);
                }
                return;
            }
            i -= size(bucket);
        }
    }

    void erase(int i) {
        siz--;
        for (int bi = 0; bi < size(a); bi++) {
            auto& bucket = a[bi];
            if (i < size(bucket)) {
                bucket.erase(begin(bucket) + i);
                if (bucket.empty()) a.erase(begin(a) + bi);
                return;
            }
            i -= size(bucket);
        }
    }

    T& access(int i) {
        for (auto& bucket : a) {
            if (i < size(bucket)) return bucket[i];
            i -= size(bucket);
        }
        assert(false);
    }
};
```

Full Dynamic Array

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
template<typename T>
struct DynArray {
    struct Node {
        T val; int pri, sz;
        Node *l, *r;
        Node(const T& v) : val(v), pri(rng()), sz(1), l(0), r(0) {}
    };

    Node* root = 0;

    static int sz(Node* t) { return t ? t->sz : 0; }
    static void pull(Node* t) { if (t) t->sz = 1 + sz(t->l) + sz(t->r); }

    static void split(Node* t, int k, Node*& l, Node*& r) {
        if (!t) { l = r = 0; return; }
        if (sz(t->l) < k) {
            split(t->r, k - sz(t->l) - 1, t->r, r);
```

```
            l = t;
        } else {
            split(t->l, k, l, t->l);
            r = t;
        }
        pull(t);
    }

    static void merge(Node*& t, Node* l, Node* r) {
        if (!l || !r) { t = l ? l : r; return; }
        if (l->pri > r->pri) {
            merge(l->r, l->r, r);
            t = l;
        } else {
            merge(r->l, l, r->l);
            t = r;
        }
        pull(t);
    }

    void insert(int pos, const T& val) {
        Node *l, *r;
        split(root, pos, l, r);
        Node* nd = new Node(val);
        merge(l, l, nd);
        merge(root, l, r);
    }

    void erase(int pos) {
        Node *l, *m, *r;
        split(root, pos, l, m);
        split(m, 1, m, r);
        delete m;
        merge(root, l, r);
    }

    T& access(int pos) {
        Node* t = root;
        while (true) {
            int left = sz(t->l);
            if (pos < left) t = t->l;
            else if (pos == left) return t->val;
            else { pos -= left + 1; t = t->r; }
        }
    }

    int size() { return sz(root); }
};
```

Count below

```
// counts how many pairs that sum <= limit
template <typename T>
ll count_below(vector<T>& v, int sz, ll Limit){
    // unique pairs such that v[i]+v[j] <= limit
    assert(is_sorted(v.begin(), v.end()));
```

```
    ll total = 0;
    for (int l = 0, r = sz-1; l < r; l++){
        while(r > l && v[l] + v[r] > Limit)
            r--;
        total += max(0, r-l);
    }
    return total;
}
```

3. Graphs and Trees

Bellman Ford

```
// Computes Single-Source Shortest Paths (SSSP) on graphs that contain NEGATIVE edge weights.
// It explicitly detects negative weight cycles reachable from the source vertex.
// - Average Time Complexity: O(E) where E is the number of edges.
// - Worst Case Time Complexity: O(V * E).
// vector<int> dist;
// bool ok = spfa(0, adj, dist);

const int INF = 1e9;
bool spfa(int s, const vector<vector<pair<int, int>>>& adj, vector<int>& d) {
    int n = adj.size();
    d.assign(n, INF);
    vector<int> cnt(n, 0);
    vector<int> inqueue(n, 0);
    queue<int> q;

    d[s] = 0;
    q.push(s);
    inqueue[s] = 1;

    while (!q.empty()) {
        int v = q.front();
        q.pop();
        inqueue[v] = 0;
        for (const auto& [to, len] : adj[v]) {
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                if (!inqueue[to]) {
                    q.push(to);
                    inqueue[to] = 1;
                    if (++cnt[to] > n) return false;  // negative cycle
                }
            }
        }
    }
    return true;
}
```

Floyd Tricks

```
void TransitiveClosure(int n, vector<vector<int>>& adj) {
    // 0 = disconnected, 1 = connected
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                adj[i][j] |= (adj[i][k] & adj[k][j]);
}

void minimax(int n, vector<vector<int>>& adj) {
    // Path such that max value on road is minimized
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
```

```
                adj[i][j] = min(adj[i][j], max(adj[i][k], adj[k][j]));
}

void maximin(int n, vector<vector<int>>& adj) {
    // Path such that min value on road is maximized
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                adj[i][j] = max(adj[i][j], min(adj[i][k], adj[k][j]));
}

void longestPathDAG(int n, vector<vector<int>>& adj) {
    // Only works for DAGs (no positive cycles)
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                adj[i][j] = max(adj[i][j], max(adj[i][k], adj[k][j]));
}

void countPaths(int n, vector<vector<int>>& adj) {
    // Floyd-Warshall for counting number of paths
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                adj[i][j] += adj[i][k] * adj[k][j];
}

bool isNegativeCycle(int n, const vector<vector<int>>& adj) {
    // run floyd first
    for (int i = 0; i < n; ++i)
        if (adj[i][i] < 0)
            return true;
    return false;
}

bool anyEffectiveCycle(int n, const vector<vector<int>>& adj,
        int src, int dest, int OO) {
    // run floyd first
    for (int i = 0; i < n; ++i)
        if (adj[i][i] < 0 && adj[src][i] < OO && adj[i][dest] < OO)
            return true;
    return false;
}
```

Topological Sort

```
queue<int> queue;
for (int i = 0; i < n; i++)
    if (in_degree[i] == 0) { queue.push(i); }

vector<int> top_sort;
while (!queue.empty()) {
    int curr = queue.front();
    queue.pop();
    top_sort.push_back(curr);
    for (int next : graph[curr]) {
```

```
        if (--in_degree[next] == 0) { queue.push(next); }
    }
}
```

Bipartite Check

```
int n;
vector<vector<int>> adj;

vector<int> side(n, -1);
bool is_bipartite = true;
queue<int> q;
for (int st = 0; st < n; ++st) {
    if (side[st] == -1) {
        q.push(st);
        side[st] = 0;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int u : adj[v]) {
                if (side[u] == -1) {
                    side[u] = side[v] ^ 1;
                    q.push(u);
                } else {
                    is_bipartite &= side[u] != side[v];
                }
            }
        }
    }
}
```

Tree Diameter

```
pair<int, int> dfs(int u, int p) {
    pair<int, int> ret = {0, u};
    for (int v : tree[u]) {
        if (v == p) continue;
        auto x = dfs(v, u);
        ret = max(ret, {x.first + 1, x.second});
    }
    return ret;
}
```

Tree

```
struct tree {
    int root = 0;
    vector<vector<int>> g;
    explicit tree(int n) : g(n) { }
    void add(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    vector<int> &operator[](int u) {
```

```
        return g[u];
    }

    int cntDfs = 0;
    vector<int> in, out, lvl, sz, top, par, seq;

    void init(int rt = 0) {
        root = rt;
        in = out = lvl = top = par = seq = vector<int>(g.size());
        sz.resize(g.size(), 1);
        par[root] = top[root] = root;
        dfs(root);
        dfs2(root);
    }

    void dfs(int u) {
        for(int &v : g[u]) {
            lvl[v] = lvl[u] + 1;
            par[v] = u;
            g[v].erase(find(g[v].begin(), g[v].end(), u));
            dfs(v);
            sz[u] += sz[v];
            if(sz[v] > sz[g[u][0]])
                swap(v, g[u][0]);
        }
    }
    void dfs2(int u) {
        in[u] = cntDfs++;
        seq[in[u]] = u;
        for(int v : g[u]) {
            top[v] = v == g[u][0]? top[u]: v;
            dfs2(v);
        }
        out[u] = cntDfs - 1;
    }

    int jump(int u, int k) {
        if(k > lvl[u]) return -1;

        int d = lvl[u] - k;
        while (lvl[top[u]] > d) {
            u = par[top[u]];
        }

        return seq[in[u] - lvl[u] + d];
    }

    bool isAncestor(int u, int v) {
        return in[u] <= in[v] && in[v] <= out[u];
    }

    int lca(int u, int v) {
        if(lvl[u] > lvl[v]) swap(u, v);
        if(isAncestor(u, v)) return u;
        while (top[u] != top[v]) {
            if (lvl[top[u]] > lvl[top[v]]) {
                u = par[top[u]];
```

```
            } else {
                v = par[top[v]];
            }
        }
        return lvl[u] < lvl[v]? u: v;
    }

    int dis(int u, int v) {
        return lvl[u] + lvl[v] - 2 * lvl[lca(u, v)];
    }
};
```

LCA

```
struct LCA {
    int n, m, LOG;
    vector<vector<int>> g;
    vector<int> euler, first, depth, lg;
    vector<vector<int>> st;

    LCA(const vector<vector<int>>& tree, int root) {
        n = tree.size();
        g = tree;
        first.assign(n, -1);
        depth.assign(n, 0);

        dfs(root, -1, 0);
        m = euler.size();

        lg.assign(m+1, 0);
        for (int i = 2; i <= m; i++)
            lg[i] = lg[i>>1] + 1;
        LOG = lg[m] + 1;

        st.assign(m, vector<int>(LOG));
        for (int i = 0; i < m; i++)
            st[i][0] = euler[i];

        for (int j = 1; j < LOG; j++) {
            for (int i = 0; i + (1<<j) <= m; i++) {
                int x = st[i][j-1], y = st[i + (1<<(j-1))][j-1];
                st[i][j] = (depth[x] < depth[y] ? x : y);
            }
        }
    }

    void dfs(int u, int p, int d) {
        if (first[u] == -1) first[u] = euler.size();
        euler.push_back(u);
        depth[u] = d;
        for (int v : g[u]) {
            if (v == p) continue;
            dfs(v, u, d+1);
            euler.push_back(u);
        }
    }
```

```
    int lca(int u, int v) {
        int L = first[u], R = first[v];
        if (L > R) swap(L, R);
        int len = R - L + 1, k = lg[len];
        int x = st[L][k], y = st[R - (1<<k) + 1][k];
        return depth[x] < depth[y] ? x : y;
    }

    int dis(int u, int v) {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }
};
```

DSU on Trees

```
void pre(int u) {
    sz[u] = 1;
    for (int &v : tree[u]) {
        tree[v].erase(find(tree[v].begin(), tree[v].end(), u));
        pre(v);
        sz[u] += sz[v];
        if (sz[v] > sz[tree[u].front()]) swap(v, tree[u].front());
    }
}

void addver(int u) {

}
void removever(int u) {

}

void addsub(int u) {
    addver(u);
    for (int v : tree[u]) addsub(v);
}
void removesub(int u) {
    removever(u);
    for (int v : tree[u]) removesub(v);
}

void dfs(int u) {
    for (int v : tree[u]) if (v != tree[u].front()) {
        dfs(v);
        removesub(v);
    }
    if (!tree[u].empty())
        dfs(tree[u].front());
    addver(u);
    for (int v : tree[u])
        if (v != tree[u].front())
            addsub(v);

    // query[u]
}
```

SCC / Strongly Connected Componenets

```cpp
// --- USAGE REQUIREMENTS ---
// 1. timer must be declared and initialized to 0 outside the function.
// 2. g is your 0-based directed adjacency list (vector<vector<int>>).
// 3. v is a 0-based array/vector of vertex weights
    (used to find the min weight per SCC).
// 4. idscc[i] will hold the 0-based SCC ID for vertex i.
// 5. cond generates the condensed DAG where each node is a distinct SCC.

// --- MODIFICATIONS FOR BRIDGES & ARTICULATION POINTS (UNDIRECTED GRAPH) ---
// 1. Update the lambda signature to pass the parent: tarj = [&](int u, int p = -1)
// 2. Add int children = 0; at the top of the lambda to
    count root children for APs.
// 3. Ignore the back-edge to the parent in the loop: if (v_ == p) continue;
// 4. You can completely remove vis, stk, mn, and idscc
    arrays if you are not extracting SCCs.

vector<int> tin(n, -1), low(n), vis(n), stk, mn, idscc(n, -1);
function<void(int)> tarj = [&](int u) { // Change to: (int u, int p = -1)
    tin[u] = low[u] = timer++;
    vis[u] = 1; // Marks u as currently in the active SCC stack
    stk.push_back(u);

    // int children = 0; // Uncomment for Articulation Points

    for (int v_ : g[u]) {
        // Uncomment to prevent traversing back to parent in undirected graphs
        // if (v_ == p) continue;

        if (tin[v_] == -1) {
            // children++; // Uncomment for Articulation Points
            tarj(v_); // Change to: tarj(v_, u)
            low[u] = min(low[u], low[v_]);

            // --- BRIDGE CHECK ---
            // if (low[v_] > tin[u]) { /* Edge (u, v_) is a bridge */ }

            // --- ARTICULATION POINT CHECK (NON-ROOT) ---
            // if (low[v_] >= tin[u] && p != -1) { /* Vertex u is an AP */ }

        } else if (vis[v_]) {
            // Back-edge found. v_ is already visited AND
            // still in the current stack
            low[u] = min(low[u], tin[v_]);
        }
    }

    // --- SCC EXTRACTION LOGIC ---
    // If u is the root of an SCC, pop all vertices belonging to
    // it from the stack
    if (low[u] == tin[u]) {
        int t;
        mn.push_back(1e5); // Initialize the minimum weight for this new SCC
        do {
            t = stk.back();
            stk.pop_back();
```

```
                vis[t] = 0; // Mark as removed from the active stack
                // Track min value (requires your v array)
                mn.back() = min(mn.back(), v[t]);
                // Assign the new SCC ID to vertex t
                idscc[t] = (int)mn.size() - 1;
            } while (t ^ u);
        }

        // --- ARTICULATION POINT CHECK (ROOT) ---
        // if (p == -1 && children > 1) { /* Vertex u (the root) is an AP */ }
    };

    for (int i = 0; i < n; i++)
        if (tin[i] == -1)
            tarj(i); // Change to: tarj(i, -1)

    // --- BUILD THE CONDENSED DAG ---
    vector<vector<int>> cond(mn.size());
    for (int i = 0; i < n; i++)
        for (int u : g[i])
            if (idscc[i] ^ idscc[u]) // If an edge connects two different SCCs
                cond[idscc[i]].push_back(idscc[u]);
```

WLCA

```
template<class T>
struct WLCA {
    int n, Log;
    vector<vector<int>> up;
    vector<vector<T>> val;
    vector<int> lvl;
    explicit WLCA(vector<vector<pair<int, T>>> &g, int root = 0) : n((int)g.size()),
            lvl(n), Log(__lg(n) + 1), up(n, vector<int>(Log, root)),
            val(n, vector<T>(Log)) {
        function<void(int)> dfs = [&](int u) -> void {
            for(auto [v, w] : g[u]) {
                if(v == up[u][0]) continue;
                lvl[v] = lvl[u] + 1, up[v][0] = u, val[v][0] = w;
                for(int l = 1; l < Log; l++) {
                    up[v][l] = up[up[v][l - 1]][l - 1];
                    val[v][l] = val[v][l - 1] + val[up[v][l - 1]][l - 1];
                }
                dfs(v);
            }
        };
        dfs(root);
    }
    pair<int, T> k_ancestor(int u, int k) {
        T ans;
        while(k) {
            ans = ans + val[u][__builtin_ctz(k)];
            u = up[u][__builtin_ctz(k)];
            k &= k - 1;
        }
        return {u, ans};
    }
```

```
    int lca(int u, int v) {
        if(lvl[u] < lvl[v]) swap(u, v);
        u = k_ancestor(u, lvl[u] - lvl[v]).first;
        if(u == v) return u;
        for(int l = Log - 1; l >= 0; l--) {
            if(up[u][l] ^ up[v][l]) {
                u = up[u][l], v = up[v][l];
            }
        }
        return up[u][0];
    }
    int dis(int u, int v, int l = -1) {
        if(l == -1) l = lca(u, v);
        return lvl[u] + lvl[v] - 2 * lvl[l];
    }
};
```

Rerooter

```
namespace reroot {
    const auto exclusive = [](const auto& a, const auto& base,
                              const auto& merge_into, int vertex) {
        int n = (int)a.size();
        using Aggregate = decay_t<decltype(base)>;
        vector<Aggregate> b(n, base);
        for (int bit = __lg(n); bit >= 0; --bit) {
            for (int i = n - 1; i >= 0; --i) b[i] = b[i >> 1];
            int sz = n - (n & !bit);
            for (int i = 0; i < sz; ++i) {
                int index = (i >> bit) ^ 1;
                b[index] = merge_into(b[index], a[i], vertex, i);
            }
        }
        return b;
    };
    // MergeInto : Aggregate * Value * Vertex(int) * EdgeIndex(int) -> Aggregate
    // Base : Vertex(int) -> Aggregate
    // FinalizeMerge : Aggregate * Vertex(int) * EdgeIndex(int) -> Value
    const auto rerooter = [](const auto& g, const auto& base,
                const auto& merge_into, const auto& finalize_merge) {
        int n = (int)g.size();
        using Aggregate = decay_t<decltype(base(0))>;
        using Value = decay_t<decltype(finalize_merge(base(0), 0, 0))>;
        vector<Value> root_dp(n), dp(n);
        vector<vector<Value>> edge_dp(n), redge_dp(n);

        vector<int> bfs, parent(n);
        bfs.reserve(n);
        bfs.push_back(0);
        for (int i = 0; i < n; ++i) {
            int u = bfs[i];
            for (auto v : g[u]) {
                if (parent[u] == v) continue;
                parent[v] = u;
                bfs.push_back(v);
```

```
            }
        }

        for (int i = n - 1; i >= 0; --i) {
            int u = bfs[i];
            int p_edge_index = -1;
            Aggregate aggregate = base(u);
            for (int edge_index = 0; edge_index < (int)g[u].size(); ++edge_index) {
                int v = g[u][edge_index];
                if (parent[u] == v) {
                    p_edge_index = edge_index;
                    continue;
                }
                aggregate = merge_into(aggregate, dp[v], u, edge_index);
            }
            dp[u] = finalize_merge(aggregate, u, p_edge_index);
        }

        for (auto u : bfs) {
            dp[parent[u]] = dp[u];
            edge_dp[u].reserve(g[u].size());
            for (auto v : g[u]) edge_dp[u].push_back(dp[v]);
            auto dp_exclusive = exclusive(edge_dp[u], base(u), merge_into, u);
            redge_dp[u].reserve(g[u].size());
            for (int i = 0; i < (int)dp_exclusive.size(); ++i)
                redge_dp[u].push_back(finalize_merge(dp_exclusive[i], u, i));
            root_dp[u] = finalize_merge(n > 1 ?
                merge_into(dp_exclusive[0], edge_dp[u][0], u, 0) :
                base(u), u, -1);
            for (int i = 0; i < (int)g[u].size(); ++i) {
                dp[g[u][i]] = redge_dp[u][i];
            }
        }

        return make_tuple(move(root_dp), move(edge_dp), move(redge_dp));
    };
}  // namespace reroot
// [&](Aggregate agg, Aggregate chdp, int v, int eid)
// [&](Aggregate agg, int v, int eid);
```

Dynamic Connectivity

```
struct Query {
    char t;
    int u, v;
};

struct Elem {
    int u, v, szU, cnt;
};

struct DSURollback {
    // offline : [+ a b] add edge between a, b
    //           [- a b] remove edge between a, b
    //           [?] number of connected components
    int cnt;
    stack<Elem> st;
```

```
    vector<bool> ans;
    vector<int> sz, par;
    map<int, vector<pair<int, int>>> g;

    DSURollback(int n) {
        cnt = n;
        par.resize(n + 1);
        sz.resize(n + 1, 1);
        iota(par.begin(), par.end(), 0);
    }

    void rollback(int x) {
        while (st.size() > x) {
            auto e = st.top();
            st.pop();
            cnt = e.cnt;
            sz[e.u] = e.szU;
            par[e.v] = e.v;
        }
    }

    int findSet(int u) {
        return par[u] == u ? u : findSet(par[u]);
    }

    void update(int u, int v) {
        st.push({u, v, sz[u], cnt});
        cnt--;
        par[v] = u;
        sz[u] += sz[v];
    }

    void unionSet(int u, int v) {
        u = findSet(u);
        v = findSet(v);
        if (u != v) {
            if (sz[u] < sz[v])
                swap(u, v);
            update(u, v);
        }
    }

    void solve(int x, int l, int r) {
        int cur = st.size();

        for (auto i: g[x])
            unionSet(i.first, i.second);

        if (l == r) {
            if (ans[l])
                cout << cnt << endl;
            rollback(cur);
            return;
        }
        int m = (l + r) >> 1;
        solve(x * 2, l, m);
```

```
        solve(x * 2 + 1, m + 1, r);
        rollback(cur);
    }

    void traverse(int x, int lX, int rX, int l, int r, int u, int v) {
        if (rX < l || lX > r)
            return;
        if (lX >= l && rX <= r) {
            g[x].emplace_back(u, v);
            return;
        }
        int m = (lX + rX) >> 1;
        traverse(x * 2, lX, m, l, r, u, v);
        traverse(x * 2 + 1, m + 1, rX, l, r, u, v);
    }

    void build(vector<Query> &queries) {
        int q = queries.size();
        ans.resize(q);
        map<pair<int, int>, vector<pair<int, int>>> mp;
        for (int i = 0; i < queries.size(); i++) {
            auto cur = queries[i];
            if (cur.u > cur.v)
                swap(cur.u, cur.v);
            if (cur.t == '?')
                ans[i] = 1;
            else if (cur.t == '+')
                mp[{cur.u, cur.v}].emplace_back(i, queries.size());
            else {
                mp[{cur.u, cur.v}].back().second = i - 1;
                traverse(1, 0, q - 1, mp[{cur.u, cur.v}].back().first,
                mp[{cur.u, cur.v}].back().second, cur.u, cur.v);
            }
        }

        for (auto i: mp) {
            for (auto j: i.second) {
                if (j.second == q)
                    traverse(1, 0, q - 1, j.first, q - 1, i.first.first,
                    i.first.second);
            }
        }
    }
};

void testCase() {
    int n, q;
    cin >> n >> q;
    if (!q)
        return;
    DSURollback dsu(n);
    vector<Query> queries(q);
    char t;
    int x, y;
    for (int i = 0; i < q; i++) {
        cin >> t;
```

```
        if (t == '?')
            queries[i] = {t, 0, 0};
        else {
            cin >> x >> y;
            if (y < x)
                swap(x, y);
            queries[i] = {t, x, y};
        }
    }

    dsu.build(queries);
    dsu.solve(1, 0, q - 1);
}
```

Max Flow (Dinik)

```
class Dinic { // O(n^2 * m), 0-based
   private:
    struct E {
        int to, rev;
        ll c, oc;
        ll f() { return max(oc - c, 0LL); }
    };
    vector<int> lvl, ptr, q;
    vector<vector<E>> g;
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int &i = ptr[v]; i < g[v].size(); ++i) {
            E &e = g[v][i];
            if (lvl[e.to] == lvl[v] + 1 && e.c)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p;
                    g[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }

   public:
    Dinic(int n) : lvl(n), ptr(n), q(n), g(n) {}
    void add_edge(int u, int v, ll c) { // directed
        g[u].push_back({v, (int)g[v].size(), c, c});
        g[v].push_back({u, (int)g[u].size() - 1, 0, 0});
    }
    ll calc(int s, int t) { // source to destination
        ll flow = 0;
        while (true) {
            fill(lvl.begin(), lvl.end(), 0);
            int qs = 0, qe = 0;
            lvl[q[qe++] = s] = 1;
            while (qs < qe && !lvl[t]) {
                int v = q[qs++];
                for (auto &e : g[v])
                    if (!lvl[e.to] && e.c) lvl[q[qe++] = e.to] = lvl[v] + 1;
            }
```

```
            if (!lvl[t]) break;
            fill(ptr.begin(), ptr.end(), 0);
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        }
        return flow;
    }
    void reset() { // before calling calc again
        for (auto &adj : g)
            for (auto &e : adj) e.c = e.oc;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

Max Bipartite Matching (Karp) [with building]

```
struct matching {
    int nl, nr;
    vector<vector<int>> g;
    vector<int> dis, ml, mr;
    explicit matching(int nl, int nr) : nl(nl), nr(nr), g(nl), dis(nl), ml(nl, -1),
    mr(nr, -1) { }

    void add(int l, int r) { // [i, j]
        g[l].push_back(r);
    }

    void bfs() {
        queue<int> q;
        for(int u = 0; u < nl; u++) {
            if(ml[u] == -1) q.push(u), dis[u] = 0;
            else dis[u] = -1;
        }
        while(!q.empty()) {
            int l = q.front(); q.pop();
            for(int r : g[l]) {
                if(mr[r] != -1 && dis[mr[r]] == -1)
                    q.push(mr[r]), dis[mr[r]] = dis[l] + 1;
            }
        }
    }

    bool canMatch(int l) {
        for(int r : g[l]) if(mr[r] == -1)
            return mr[r] = l, ml[l] = r, true;
        for(int r : g[l]) if(dis[l] + 1 == dis[mr[r]] && canMatch(mr[r]))
            return mr[r] = l, ml[l] = r, true;
        return false;
    }

    int maxMatch() {
        int ans = 0, turn = 1;
        while(turn) {
            bfs(), turn = 0;
            for(int l = 0; l < nl; l++) if(ml[l] == -1)
                turn += canMatch(l);
            ans += turn;
```

```
        }
        return ans;
    }

    pair<vector<int>, vector<int>> minCover() {
        vector<int> L, R;
        for (int u = 0; u < nl; ++u) {
            if(dis[u] == -1) L.push_back(u);
            else if(ml[u] != -1) R.push_back(ml[u]);
        }
        return {L, R};
    }
};
```

4. Math and Number Theory

Math

```
Stirling numbers of the first kind : the number of permutations of n elements with k disjoint
S(n,k) = n * S(n-1,k) + S(n-1,k-1)
Sum k = 0 -> n of S(n,k) = n!

Stirling numbers of the second kind : the number of ways to partition
a set of n elements into k nonempty subsets
S(n,k) = k * S(n-1,k) + S(n-1,k-1)
Sum k = 0 -> n of S(n,k) = Bn

Bell numbers : the possible partitions of a set into nonempty subsets
Sum k = 0 -> n-1 of n-1Ck * Bk = Bn

Sum of first n even numbers : n*(n+1)
Sum of first n odd numbers : n*n

Sum of squares of first n numbers:
n*(n+1)*(2*n+1)/6
Sum of squares of first n even numbers:
2*n*(n+1)*(2*n+1)/3
Sum of squares of first n odd numbers:
n*(2*n+1)*(2*n-1)/3

Number of ways to pick equal number of elements from two sets : (n+m)C(m)

Sum of phi(d) for all d | n is equal to n.
Number of pairs (x,y) that satisfy x+y=n and gcd(x,y)=1 is phi(n).

Sum(nCk) for k [0,n] = 2^n
Sum(mCk) for all m [0,n] = (n+1)C(k+1)
Sum((n+k)Ck) for all k [0,m] = (n+m+1)C(m)
Sum((nCk)^2) for all k [0,n] = (2*n)Cn
Sum(i*nCi) for all i [1,n] = n*2^(n-1)
Sum((n-i)Ci) for all i [0,n] = F(n+1)
Number of arrays with size n and sum m = (n-1+m)C(m) = (n-1+m)C(n-1)

P(A|B) is the probability of event A given that event B happened.
P(A&B) is the probability of events A and B happening.

P(A|B) = (P(B|A) * P(A) ) / P(B)
P(A|B) = P(A&B) / P(B)

Divisibility:
2 if the rightmost digit is divisible by 2
3 if the sum of the digits is divisible by 3
4 if the number formed by the last two digits is divisible by 4
5 if the rightmost digit is 0 or 5
6 if it is divisible by 2 and 3
7 if The number formed by all digits except the right-most digit -
    (2 * right-most digit) is divisible by 7
8 if the number formed by the last 3 digits is divisible by 8
9 if the sum of the digits is divisible by 9

-Getting half of the binomial expansion (Only odd indices or only even indices)
```

```
    by using (a+b)^n and (a-b)^n and adding both of them

To solve quadratic equation a*x^2 + b*x + c = 0
x = -b (+-)sqrt(b^2-4*a*c) / (2 * a)

-Sum of geometric series ar^i (i from 0 to n) = a(1 - r^n) / (1 - r)
-Sum of geometric series ar^i (i from 0 to infinity) = a / (1 - r)

- XOR from 1 to n = n%4 [n, 1, n+1, 0]

- n can be sum of 2 squares if n has no p^k [p = 3 % 4, k is odd]
    in its prime factorization
- n can be sum of 3 squares if n is not in form [4^a (8b + 7)]
- n always can be sum of >= 4 squares (remaining are zeros)

=============== FADY THE GOOOOOAAT ===============
 sum of divisors
 prime^power * prime2^power2 * ...

 ((prime^(power + 1) - 1) / (prime - 1)) * ((prime2^(power2 + 1) - 1) / (prime2 - 1)) * ...
==================================================
 a % m == b
 a and m not coprime
 g = gcd(a, m)
 (a / g) % (m / g) = b / g

 a^x % m == b
 a and m not coprime
 g = gcd(a, m)
 (a^(x-1) * (a / g)) % (m / g) = b / g
 ================================================================
 a^(power%phi(m)) % m;
 ================================================================
 count balanced brackets
 r=n/2  ||  or r = number of opened brackets
 nCr(n, r) - nCr(n, r-1)
 ================================================================
 // different n*n grids whose each square have m colors
 // if possible to rotate one of them so that they look the same then they same
 t = n * n;
 total = (fp(m, t)
     + fp(m, (t + 1) / 2)
     + 2 * fp(m, (t / 4) + (n % 2))) % mod;
 total = mul(total, fp(4, mod - 2));
 ================================================================
 biggest divisors
 735134400 1344 => 2^6 3^3 5^2 7 11 13 17
 73513440 768
 ================================================================
 for (int x = mask; x > 0; x = (x - 1) & mask)
 get all x such that mask = mask | x
 ================================================================
 sum from 1 to n: i * nCr(n, i) = n * (1LL << (n - 1))
 sum of odd between [1, n]  = ((n+1) / 2)^2
 sum of even between [1, n] = (n/2) * (n/2 + 1)

ll sum_total(ll l, ll r) { // sum [l, r]
```

```
    return (r - l + 1) * (l + r) / 2;
}

ll sum_even(ll l, ll r) { // sum even [l, r]
    if (l % 2 != 0) ++l;
    if (r % 2 != 0) --r;
    if (l > r) return 0;
    ll n = (r - l) / 2 + 1;
    return n * (l + r) / 2;
}

ll sum_odd(ll l, ll r) { // sum odd [l, r]
    if (l % 2 == 0) ++l;
    if (r % 2 == 0) --r;
    if (l > r) return 0;
    ll n = (r - l) / 2 + 1;
    return n * (l + r) / 2;
}

ll arithm1(ll l, ll r, ll a, ll d) { // [l, r] starting a and diff d
    if (d == 0) return (a >= l && a <= r) ? a : 0;
    ll n1 = ((l - a + d - (d > 0 ? 1 : -1)) / d);
    ll first = a + n1 * d;
    if ((d > 0 && first > r)
        || (d < 0 && first < r))
            return 0;
    ll n2 = ((r - a) / d);
    ll last = a + n2 * d;
    ll n = (n2 - n1 + 1);
    return n * (first + last) / 2;
}

ll arithm2(ll a, ll d, ll n) { // starting a, diff d, n terms
    return n * (2 * a + (n - 1) * d) / 2;
}
 */

ll phi(ll x) { // sqrt(x)
    ll ans = x;
    for(ll i = 2; i * i <= x; i++) {
        if(x % i == 0) {
            while(x % i == 0) x /= i;
            ans -= ans / i;
        }
    }
    if(x > 1) ans -= ans / x;
    return ans;
}

array<ll, 2> CRT(ll a1, ll m1, ll a2, ll m2) {
    // x = a1 % m1, x = a2 % m2
    a1 %= m1, a2 %= m2;
    auto [g, q1, q2] = eGcd(m1, -m2);
    if ((a2 - a1) % g) return {-1, -1};
    ll lcm = m1 / g * m2;
    ll m = m2 / g;
```

```cpp
    q1 = (a2 - a1) / g % m * q1 % m;
    ll res = (a1 + m1 * q1) % lcm;
    if (res < 0) res += lcm;
    return {res, lcm};
}

ll BSGS(ll a, ll b, ll p) { // a^x = b (mod p)
    a %= p, b %= p;
    if(b == 1) return 0;
    if(a == 0) return b == 0? 1: -1;
    int add = 0;
    ll g, tmp = 1;
    while ((g = gcd(a, p)) > 1) {
        if(b % g) return -1;
        p /= g, b /= g, tmp = tmp * (a / g) % p, ++add;
        if(tmp == b) return add;
    }
    b = b * modInv(tmp, p) % p;
    int n = (int)sqrtl(p) + 1;
    unordered_map<ll, int> mp;
    for (ll q = 0, cur = 1; q <= n; ++q)
        mp.emplace(cur, q), cur = cur * a % p;
    ll an = 1;
    for (ll i = 0; i < n; ++i) an = an * a % p;
    an = modInv(an, p);
    for (ll i = 0, cur = b; i <= n; ++i) {
        auto it = mp.find(cur);
        if(it != mp.end()) return i * n + it->second + add;
        cur = cur * an % p;
    }
    return -1;
}

int sumNPowerM(int n, int m) { // 1^m + 2^m ... n^m
    int k = m + 3;
    vector<int> res(k);
    for(int i = 1; i < k; i++) res[i] = (res[i - 1] + fp(i, m)) % mod;
    if(n < k) return res[n];
    int facK = k;
    vector<int> p(k); p[0] = 1;
    for(int i = 1; i < k; i++) {
        p[i] = int(p[i - 1] * 1LL * (n - i) % mod);
        facK = int(facK * 1LL * i % mod);
    }
    vector<int> inv(k + 1), s(k + 1);
    inv[k] = fp(facK, mod - 2), s[k] = 1;
    for(int i = k - 1; i >= 0; i--) {
        s[i] = int(s[i + 1] * 1LL * (n - i) % mod);
        inv[i] = int(inv[i + 1] * 1LL * (i + 1) % mod);
    }
    int ans = 0;
    for(int i = 1; i < k; i++) {
        int cur = int(res[i] * 1LL * p[i - 1] % mod * s[i + 1] % mod *
            inv[i - 1] % mod * inv[k - i - 1] % mod);
        if((k - i + 1) & 1) cur = (mod - cur) % mod;
        ans = (ans + cur) % mod;
```

```
    }
    return ans;
}
```

ModInt

```
template <int M> struct ModInt {
    template <class T> T binpow(T a, ll b) {
        T res = 1;
        while (b) { if (b & 1) res *= a; a *= a, b >>= 1; }
        return res;
    }
    int v;
    ModInt() : v(0) {}
    ModInt(ll v_) {
        v = v_ % M;
        if (v < 0) v += M;
    }

    bool operator==(ModInt o) const { return v == o.v; };
    bool operator!=(ModInt o) const { return !(*this == o); };

    // ModInt add(ModInt a, ModInt b) { return ((a.v + b.v % M)+M)%M;}
    // ModInt sub(ModInt a, ModInt b) { return add(a, -b); }
    // ModInt mul(ModInt a, ModInt b) { return 1ll * a.v * b.v % M; }
    // ModInt div(ModInt a, ModInt b) { return mul(a, binpow(b, M-2)); }

    ModInt &operator+=(ModInt o) { v = (v + o.v) % M; return *this; }
    ModInt &operator-=(ModInt o) { v = (v - o.v + M) % M; return *this; }
    ModInt &operator*=(ModInt o) { v = 1ll * v * o.v % M; return *this; }
    ModInt &operator/=(ModInt o) { return (*this *= binpow(o, M - 2)); }

    friend ModInt operator+(ModInt a, ModInt b) { return a += b; }
    friend ModInt operator-(ModInt a, ModInt b) { return a -= b; }
    friend ModInt operator*(ModInt a, ModInt b) { return a *= b; }
    friend ModInt operator/(ModInt a, ModInt b) { return a /= b; }
    ModInt operator-() { return 0 - *this; }

    friend istream &operator>>(istream &is, ModInt &a) {
        ll x; is >> x;
        a = ModInt(x);
        return is;
    }
    friend ostream &operator<<(ostream &os, ModInt a) { return os << a.v; }
    friend string to_string(ModInt a) { return to_string(a.v); }
};
using mint = ModInt<>;
```

Sieve / PHI up to n / 2D gcd()

```
const int NS = 1e7;
const int NP = (NS/log(NS)) * (1 + 1.28 / log(NS));
int prSz;
int spf[NS], prm[NP];

```

```cpp
auto pre_Sieve = []() {
    for (int i = 2; i < NS; i++){
        if(!spf[i]) spf[i] = prm[prSz++] = i;
        for(int j = 0; i * prm[j] < NS; j++) {
            spf[i * prm[j]] = prm[j];
            if(spf[i] == prm[j]) break;
        }
    }
    return 0;
}();

auto factors(int n) {
    vector<array<int, 2>> res;
    if(n < 2) return res;
    int p = spf[n];
    while(p > 1) {
        res.push_back({p, 0});
        while(n % p == 0) n /= p, res.back()[1]++;
        p = spf[n];
    }
    return res;
}

auto getDivisors(int _n) {
    auto _fac = factors(_n);
    int cnt = 1;
    for(auto [pr, pw] : _fac) cnt *= pw + 1;
    vector<int> res(1, 1); res.reserve(cnt);

    for(auto [pr, pw] : _fac)
        for(int i = int(res.size()) - 1; i >= 0; i--)
            for(int b = pr, j = 0; j < pw; j++, b *= pr)
                res.push_back(res[i] * b);
    sort(res.begin(), res.end());
    return res;
}

bool isPrime(ll n) {
    if(n < 4) return n > 1;
    if(n % 2 == 0 || n % 3 == 0) return false;
    for (ll i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    return true;
}

void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
```

```
    }
}

// 2d gcd in n^2
vector<vector<int>> GCD(N, vector<int>(N, 1));
for(int d = 2; d < N; ++d)
    for(int i = d; i < N; i += d)
        for(int j = d; j < N; j += d)
            GC[i][j] = d;
```

Sieve up to 1e9

```
vector<int> sieve(const int BN, const int Q = 17, const int L = 1 << 15) {
    using u = uint32_t; using u8 = uint8_t;
    static const u rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
    struct P {
        u p;
        u pos[8];
    };

    const u v = sqrt(BN), vv = sqrt(v);
    vector<bool> isp(v + 1, true);
    for (u i = 2; i <= vv; ++i)
        if (isp[i])
            for (u j = i * i; j <= v; j += i) isp[j] = false;

    const u rsize = BN > 60184 ? BN / (log(BN) - 1.1) :
        max(1.0, BN / (log(BN) - 1.11)) + 1 + 30;
    vector<int> primes = {2, 3, 5}; u psize = 3;
    primes.resize(rsize); vector<P> sprimes;

    u pbeg = 0, prod = 1;
    for (u p = 7; p <= v; ++p) {
        if (!isp[p]) continue;
        if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
        P pp = {p, {}};
        for (int t = 0; t < 8; ++t) {
            u j = p <= Q ? p : p * p;
            while (j % 30 != rs[t]) j += p << 1;
            pp.pos[t] = j / 30;
        }
        sprimes.push_back(pp);
    }

    vector<u8> pre(prod, 0xFF);
    for (size_t pi = 0; pi < pbeg; ++pi) {
        auto &pp = sprimes[pi];
        const u p = pp.p;
        for (int t = 0; t < 8; ++t) {
            const u8 m = ~(1 << t);
            for (u i = pp.pos[t]; i < prod; i += p) pre[i] &= m;
        }
    }

    const u block_size = (L + prod - 1) / prod * prod;
    vector<u8> block(block_size);
```

```cpp
    u8* __restrict pblock = block.data();
    const u M = (BN + 29) / 30;

    for (u beg = 0; beg < M; beg += block_size, pblock -= block_size) {
        u end = min(M, beg + block_size);

        for (u i = beg; i < end; i += prod)
            memcpy(pblock + i, pre.data(), min(prod, end - i));
        if (beg == 0) pblock[0] &= 0xFE;

        for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
            auto &pp = sprimes[pi];
            const u p = pp.p;
            #pragma GCC unroll 8
            for (int t = 0; t < 8; ++t) {
                u i = pp.pos[t];
                const u8 m = ~(1 << t);
                for (; i < end; i += p) pblock[i] &= m;
                pp.pos[t] = i;
            }
        }

        for (u i = beg; i < end; ++i)
            for (u m = pblock[i]; m > 0; m &= m - 1)
                primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
    }

    while (psize > 0 && primes[psize - 1] > BN) --psize;
    primes.resize(psize); return primes;
}
```

Egcd, Linear Diaphontine

```cpp
array<ll, 3> eGcd(ll a, ll b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = eGcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

ax0 + by0 = g ==> ax + by = c
x0 *= c/g, y0 *= c/g

all solutions:
           x = x0 + k * (b/g),
           y = y0 - k * (a/g);

int ceildiv(int a, int b) {
    if ((a ^ b) >= 0) return (a + b - 1) / b;
    else return a / b;
}

int flordiv(int a, int b) {
    if ((a ^ b) >= 0) return a / b;
    else return (a - b + 1) / b;
}

```

```
non-negative solution:
        k >= ceildiv(-x*g,b)
        k <= flordiv(x*g, a)

bool havenonnegsol(int a, int b, int c, int& x, int& y) {
    int g = egcd(a, b, x, y);
    x *= c/g;
    y *= c/g;
    int l1 = ceildiv(-x*g, b);
    int l2 = flordiv(y*g, a);
    return l1 <= l2;
}

// MOD INV
ll modInv(ll a, ll m) {
    ll x = 1, x1 = 0, q, t, b = m;
    while(b) {
        q = a / b;
        a -= q * b, t = a, a = b, b = t;
        x -= q * x1, t = x, x = x1, x1 = t;
    }
    assert(a == 1);
    return (x + m) % m;
}
```

CRT

```
using T = __int128;
// ax + by = __gcd(a, b)
// returns __gcd(a, b)
T extended_euclid(T a, T b, T &x, T &y) {
  T xx = y = 0;
  T yy = x = 1;
  while (b) {
    T q = a / b;
    T t = b; b = a % b; a = t;
    t = xx; xx = x - q * xx; x = t;
    t = yy; yy = y - q * yy; y = t;
  }
  return a;
}
// finds x such that x % m1 = a1, x % m2 = a2. m1 and m2 may not be coprime
// here, x is unique modulo m = lcm(m1, m2). returns (x, m). on failure, m = -1.
pair<T, T> CRT(T a1, T m1, T a2, T m2) {
  T p, q;
  T g = extended_euclid(m1, m2, p, q);
  if (a1 % g != a2 % g) return make_pair(0, -1);
  T m = m1 / g * m2;
  p = (p % m + m) % m;
  q = (q % m + m) % m;
  return make_pair((p * a2 % m * (m1 / g) % m +
                  q * a1 % m * (m2 / g) % m) %  m, m);
}
```

MillerRabin

```
ll binpower(ll base, ll e, ll mod) {
    ll result = 1;
    base %= mod;
    while (e) {
        if (e & 1)
            result = (__uint128_t)result * base % mod;
        base = (__uint128_t)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(ll n, ll a, ll d, int s) {
    ll x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (__uint128_t)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};

bool MillerRabin(long long n) {
    if (n < 2)
        return false;

    vector<int> primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    for (auto a : primes)
        if (n % a == 0)
            return false;

    int r = 0;
    long long d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }

    for (int a : primes) {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
            return false;
    }
    return true;
}
```

Number of Divisors up to 1e18

```
namespace countdivisors {
    // for one second: 5000 1e18, 100000 1e9, 200000 1e6
    using ull = unsigned long long;
    using u128 = __uint128_t;
    static mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
    u128 mul128(ull a, ull b, ull m) { return (u128)a * b % m; }
```

```
ull mul_mod(ull a, ull b, ull m) { return (ull)mul128(a, b, m); }
ull pow_mod(ull a, ull d, ull m) {
    ull r = 1;
    while (d) {
        if (d & 1) r = mul_mod(r, a, m);
        a = mul_mod(a, a, m);
        d >>= 1;
    }
    return r;
}
bool isPrime(ull n) {
    if (n < 2) return false;
    for (ull p : vector<ull>({2ULL,3,5,7,11,13,17,19,23,29,31,37}))
        if (n % p == 0) return n == p;
    ull d = n - 1, s = 0;
    while (!(d & 1)) d >>= 1, ++s;
    for (ull a : vector<ull>({2ULL,325,9375,28178,450775,9780504,1795265022})) {
        if (a % n == 0) continue;
        ull x = pow_mod(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool comp = true;
        for (ull r = 1; r < s; ++r) {
            x = mul_mod(x, x, n);
            if (x == n - 1) { comp = false; break; }
        }
        if (comp) return false;
    }
    return true;
}
ull pollards_rho(ull n) {
    if (n % 2 == 0) return 2;
    while (true) {
        ull c = uniform_int_distribution<ull>(1, n - 1)(rnd);
        auto f = [&](ull x){ return (mul_mod(x, x, n) + c) % n; };
        ull x = rnd() % n, y = x, d = 1;
        while (d == 1) {
            x = f(x); y = f(f(y));
            d = gcd<ull>(x > y ? x - y : y - x, n);
        }
        if (d < n) return d;
    }
}
void factor(ull n, map<ull,int>& cnt) {
    if (n < 2) return;
    if (isPrime(n)) { cnt[n]++; return; }
    ull d = pollards_rho(n);
    factor(d, cnt);
    factor(n/d, cnt);
}
ull ans(ull n) {
    map<ull,int> cnt;
    factor(n, cnt);
    ull res = 1;
    for (auto [p, e] : cnt) res *= (e + 1);
    return res;
}
```

```
}
```

n mod 1 + n mod 2 + n mod 3 + ………. + n mod m

```
// n and m tends to(1e13) => time complexity(sqrt(n))
void solve(int idx){
    int k=n/idx;
    int st=n%idx%mod;
    int l=n/(k+1);
    int len=(idx-l)%mod;
    ans=(ans + st*len%mod + k%mod*(len*(len-1)/2%mod)%mod)%mod;
    if(l) solve(l);
}
```

n-th Fib Number

```
int fast_Fibonacci(int n) {
    int i = 1, h = 1, j = 0, k = 0, t;
    while (n > 0) {
        if (n % 2 == 1)
            t = j * h, j = i * h + j * k + t, i = i * k + t;
        t = h * h, h = 2 * k * h + t, k = k * k + t, n = n / 2;
    }
    return j;
}
```

Long Division

```
int a = 23, b = 5, n = 10;
vector<int> s;
for (int i = 0; i < n; i++) {
    unsigned long long k = a / b; // Integer division: gets the next digit
    a -= b * k;                   // Remove the part already represented
    a *= 10;                      // Shift remainder left (for next digit)
    s.push_back(k);              // Store the digit
}
```

Floor Values

```
// code to get all different values of floor(n/i)
for (ll l = 1, r = 1; (n/l); l = r + 1) { // O(sqrt)
    r = (n/(n/l));
    // q = (n/l), process the range [l, r]
}
```

Combinatorics

```
namespace combinatorics {
    vector<mint> fac_(1, 1), inv_(1, 1);
    void build(int N) {
        N += 10;
        fac_.assign(N, 1);
        inv_.assign(N, 1);
```

```
        for (int i = 2; i < N; i++)
            fac_[i] = fac_[i-1] * i;
        inv_[N-1] = 1/fac_[N-1];
        for (int i = N-2; i > 1; i--)
            inv_[i] = inv_[i+1] * (i+1);
    }
    inline mint nCr(int n, int r) {
        if(n < 0 || r < 0 || r > n) return 0;
        return fac_[n] * inv_[r] * inv_[n - r];
    }

    inline mint nCr(int64_t n, int64_t r) {
        if(n < 0 || r < 0 || r > n) return 0;
        r = min<int64_t>(r, n - r);
        mint ans = inv_[r];
        for(int64_t i = n - r + 1; i <= n; i++) ans = ans * i;
        return ans;
    }
    inline mint nPr(int n, int r) {
        if(n < 0 || r < 0 || r > n) return 0;
        return fac_[n] * inv_[n - r];
    }
    inline mint stars_and_bars(int n, int r){
        return nCr(n + r - 1, r - 1);
    }
    inline mint catalan(int n) {
        if(n < 0) return 0;
        return fac_[2 * n] * inv_[n] * inv_[n + 1];
    }
}
// using namespace combinatorics;
auto pre = []() { combinatorics::build(); return 0; }();
```

nCr, nPr without precomputation

```
ll nCr(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    r = min(r, n-r);
    ll ret = 1;
    for (int i = 0; i < r; i++)
        ret = ret * (n-i) / (i+1);
    return ret;
}
ll nPr(int n, int r) {
    if (r < 0 || r > n) return 0;
    ll ret = 1;
    for (int i = 0; i < r; ++i) {
        ret *= (n - i);
    }
    return ret;
}
```

NCR table

```
void preprocess_nCr() {
```

```
    for (int n = 0; n < N; n++) {
        C[n][0] = C[n][n] = 1;
        for (int r = 1; r < n; r++) {
            C[n][r] = (C[n - 1][r - 1] + C[n - 1][r]) % MOD;
        }
    }
}
```

Catalan numbers

```
const int MOD = ....
const int MAX = ....
int catalan[MAX];
void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=n; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1])% MOD;
            if (catalan[i] >= MOD)
                catalan[i] -= MOD;
        }
    }
}

// 1- Number of correct bracket sequence consisting of n opening and n
closing brackets.

// 2- The number of rooted full binary trees with n+1 leaves (vertices
are not numbered).
// A rooted binary tree is full if every vertex has
either two children or no children.

// 3- The number of ways to completely parenthesize n+1 factors.

// 4- The number of triangulations of a convex polygon with n+2 sides
// (i.e. the number of partitions of polygon into disjoint triangles by
using the diagonals).

// 5- The number of ways to connect the 2n points on a circle to form n
disjoint chords.

// 6- The number of non-isomorphic full binary trees with n internal
nodes (i.e. nodes having at least one son).

// 7- The number of monotonic lattice paths from point (0,0) to point
(n,n) in a square lattice of size nxn,
// which do not pass above the main diagonal (i.e. connecting (0,0) to
(n,n)).

// 8- Number of permutations of length n that can be stack sorted
// (i.e. it can be shown that the rearrangement is stack sorted if and
only if there is no such index i<j<k, such that ak<ai<aj).

// 9- The number of non-crossing partitions of a set of n elements.

// 10- The number of ways to cover the ladder 1..n using n rectangles
```

Matrix Exponentiation

```
template <typename T>
struct Matrix {
    int n, m;
    vector<T> mat;

    Matrix(int n, int m) : n(n), m(m) {
        mat.assign(n * m, 0);
    }
    Matrix(int n) : n(n), m(n) {
        mat.assign(n * n, 0);
    }

    void identity() {
        assert(n == m);
        fill(mat.begin(), mat.end(), 0);
        for (int i = 0; i < n; i++) {
            mat[i * m + i] = 1;
        }
    }

    T* operator[](int r) { return &mat[r * m]; }
    const T* operator[](int r) const { return &mat[r * m]; }

    Matrix operator*(const Matrix& o) const {
        assert(m == o.n);
        Matrix res(n, o.m);

        for (int i = 0; i < n; i++) {
            for (int k = 0; k < m; k++) {
                T val = mat[i * m + k]; if (val == 0) continue;
                for (int j = 0; j < o.m; j++) {
                    res[i][j] = (res[i][j] + 1ll * val * o[k][j]) % mod;
                    // res[i][j] = res[i][j] + val * o[k][j];
                }
            }
        }
        return res;
    }

    Matrix operator+(const Matrix& o) const {
        assert(o.n == n && o.m == m);
        Matrix ret(n, m);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) {
                ret[i][j] = ((*this)[i][j] + o[i][j]) % mod;
                // ret[i][j] = (*this)[i][j] + o[i][j];
            }
        return ret;
    }

    Matrix pow(ll k) const {
        assert(n == m);
        Matrix res(n), base = *this;
        res.identity();
        while (k) {
```

```
            if (k & 1) res = res * base;
            base = base * base;
            k >>= 1;
        }
        return res;
    }
};
```

K-th permutation

```
ll fac[21];
void preFac() {
    fac[0] = 1;
    for(int i = 1; i <= 20; ++i)
        fac[i] = fac[i - 1] * i;
}

vector<int> kth_permutation(int n, ll k) { // n^2
    vector<int> a(n), ans;
    iota(a.begin(), a.end(), 1);
    for (int i = n; i >= 1; i--) {
        ll f = fac[i - 1];
        ans.push_back(a[k / f]);
        a.erase(a.begin() + k / f);
        k %= f;
    }
    return ans;
}
```

Permutation Index

```
ll permutation_index(vector<int>& p) { // n^2
    int n = int(p.size());
    vector<int> a(n);
    iota(a.begin(), a.end(), 1);

    ll k = 0;
    for (int i = 0; i < n; ++i) {
        int j = int(find(a.begin(), a.end(), p[i]) - a.begin());
        k += j * fac[n - 1 - i];
        a.erase(a.begin() + j);
    }
    return k;
}
```

5. Strings

Rolling Hash

```
using u64 = uint64_t;
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count() ^
    (uintptr_t)make_unique<char>().get());

struct hash61 {
    static const u64 md = (1LL << 61) - 1;
    inline static u64 step = (md >> 2) + rng() % (md >> 1);
    inline static vector<u64> pw = {1};

    int n;
    vector<u64> pref, suff;

    u64 add(u64 a, u64 b) const { return (a += b) >= md ? a - md : a; }
    u64 sub(u64 a, u64 b) const { return (a += md - b) >= md ? a - md : a; }
    u64 mul(u64 a, u64 b) const { return (unsigned __int128)a * b % md; }

    template<class T>
    hash61(const T& s) : n(s.size()), pref(n + 1), suff(n + 1) {
        while (pw.size() <= n) pw.push_back(mul(pw.back(), step));
        pref[0] = suff[n] = 1;
        for (int i = 0; i < n; i++) pref[i + 1] =
            add(mul(pref[i], step), s[i]);
        for (int i = n - 1; i >= 0; i--) suff[i] =
            add(mul(suff[i + 1], step), s[i]);
    }

    u64 operator()(int l, int r) const { return sub(pref[r + 1],
        mul(pref[l], pw[r - l + 1])); }
    u64 rev(int l, int r) const { return sub(suff[l],
        mul(suff[r + 1], pw[r - l + 1])); }
};

struct hash61 {
    static const u64 md = (1LL << 61) - 1;
    inline static u64 step = (md >> 2) + rng() % (md >> 1);

    inline static u64 power(u64 b, u64 e) {
        u64 r = 1;
        while (e) {
            if (e & 1) r = (unsigned __int128)r * b % md;
            b = (unsigned __int128)b * b % md;
            e >>= 1;
        }
        return r;
    }

    inline static u64 inv_step = power(step, md - 2);
    inline static vector<u64> pw = {1}, ipw = {1};

    int n = 0;
    vector<u64> pref = {1}, suff = {0};

    u64 add(u64 a, u64 b) const { return (a += b) >= md ? a - md : a; }
```

```
    u64 sub(u64 a, u64 b) const { return (a += md - b) >= md ? a - md : a; }
    u64 mul(u64 a, u64 b) const { return (unsigned __int128)a * b % md; }

    hash61() {}
    template<class T> hash61(const T& s) { for (auto c : s) push_back(c); }

    void push_back(u64 c) {
        while (pw.size() <= n + 1) {
            pw.push_back(mul(pw.back(), step));
            ipw.push_back(mul(ipw.back(), inv_step));
        }
        pref.push_back(add(mul(pref.back(), step), c));
        suff.push_back(add(suff.back(), mul(c, pw[n])));
        n++;
    }

    void pop_back() {
        if (!n) return;
        pref.pop_back();
        suff.pop_back();
        n--;
    }

    u64 operator()(int l, int r) const { return sub(pref[r + 1], mul(pref[l],
        pw[r - l + 1])); }
    u64 rev(int l, int r) const { return mul(sub(suff[r + 1], suff[l]), ipw[l]); }
};
```

Tree Hash (rooted/unrooted)

```
// tree hash
using u64 = uint64_t;
mt19937_64 rng = []{
    u64 time_entropy = chrono::steady_clock::now().time_since_epoch().count();
    u64 memory_entropy = (uintptr_t)make_unique<char>().get();
    seed_seq ss{time_entropy, memory_entropy};
    return mt19937_64(ss);
}();
u64 SEED = rng();
u64 mix(u64 x) {
    x += SEED + 0x9e3779b97f4a7c15;
    x = (x ^ x >> 30) * 0xbf58476d1ce4e5b9;
    x = (x ^ x >> 27) * 0x94d049bb133111eb;
    return x ^ x >> 31;
}
u64 treehash(int u, int p, vector<vector<int>>& t) {
    u64 ret = 1;
    // do ret = ret * P + mix() if the order is important
    for (int v : t[u]) if (v ^ p)
        ret += mix(treehash(v, u, t));
    return ret;
}
// unrooted: find centroids and treat them as roots
u64 utreehash(vector<vector<int>>& t) {
    int n = t.size();
    vector<int> sz(n), cents;
```

```
    function<void(int, int)> dfs1 = [&](int u, int p) {
        sz[u] = 1;
        bool can = 1;
        for (int v : t[u]) if (v ^ p) {
            dfs1(v, u);
            if (sz[v] * 2 > n) can = 0;
            sz[u] += sz[v];
        }
        if (n - sz[u] > n/2) can = 0;
        if (can) cents.push_back(u);
    };
    dfs1(0, 0);
    return min(treehash(cents.front(), -1, t), treehash(cents.back(), -1, t));
}
```

Z-Algo

```
vector<int> z_algo(string s) {
    vector<int> z(s.size());
    for(int i = 1, l = 0, r = 0; i < s.size(); i++) {
        if(i < r) z[i] = min(r - i, z[i - l]);
        while(i + z[i] < s.size() && s[z[i]] == s[z[i] + i]) z[i]++;
        if(i + z[i] > r) r = i + z[i], l = i;
    }
    return z;
}
```

Largest Lexicographical Substring

```
string largestLexSubstring(const string &s) {
    int n = int(s.size());
    int i = 0, j = 1, k = 0;

    while (j + k < n) {
        if (s[i + k] == s[j + k]) k++;
        else if (s[i + k] < s[j + k])
            /* change it to > if you want lowest */
            i = max(i + k + 1, j), j = i + 1, k = 0;
        else j = j + k + 1, k = 0;
    }

    return s.substr(i);
}
```

Manacher

```
auto manacher(const string &t) {
    string s = "%#";
    s.reserve(t.size() * 2 + 3);
    for(char c : t) s += c + "#"s;
    s += '$';
    // t = aabaacaabaa -> s = %#a#a#b#a#a#c#a#a#b#a#a#$

    vector<int> res(s.size());
    for(int i = 1, l = 1, r = 1; i < s.size(); i++) {
```

```
        res[i] = max(0, min(r - i, res[l + r - i]));
        while(s[i + res[i]] == s[i - res[i]]) res[i]++;
        if(i + res[i] > r) {
            l = i - res[i];
            r = i + res[i];
        }
    }
    for(auto &i : res) i--;
    return vector(res.begin() + 2, res.end() - 2); // a#a#b#a#a#c#a#a#b#a#a
    // get max odd len = res[2 * i]; aba -> i = b
    // get max even len = res[2 * i + 1]; abba -> i = first b
}
```

Aho Corasick Algorithm

```
namespace corasick {
    const int N = ++++++;
    const int SIGMA = ++++++;

    int nxt[N][SIGMA], fail_link[N], dict_link[N], match_idx[N], nodes;

    void init() {
        nodes = fail_link[0] = dict_link[0] = 0;
        memset(nxt[0], 0, sizeof(nxt[0]));
        match_idx[0] = -1;
        nodes++;
    }

    int create_node() {
        memset(nxt[nodes], 0, sizeof(nxt[nodes]));
        fail_link[nodes] = dict_link[nodes] = 0;
        match_idx[nodes] = -1;
        return nodes++;
    }

    int insert(const string& pattern, int id) {
        int cur = 0;
        for (char c : pattern) {
            if (!nxt[cur][c]) {
                nxt[cur][c] = create_node();
            }
            cur = nxt[cur][c];
        }
        if (~match_idx[cur]) return match_idx[cur];
        return match_idx[cur] = id;
    }

    void build() {
        queue<int> q;

        for (int c = 0; c < SIGMA; ++c) {
            int chi = nxt[0][c];
            if (chi) {
                fail_link[chi] = dict_link[chi] = 0;
                q.push(chi);
            }
        }
```

```

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            for (int c = 0; c < SIGMA; ++c) {
                int chi = nxt[u][c];

                if (chi) {
                    fail_link[chi] = nxt[fail_link[u]][c];

                    int fail = fail_link[chi];
                    if (match_idx[fail] != -1)
                        dict_link[chi] = fail;
                    else
                        dict_link[chi] = dict_link[fail];
                    q.push(chi);
                } else
                    nxt[u][c] = nxt[fail_link[u]][c];
            }
        }
    }

    vector<vector<int>> search(const string& text, const vector<int>& lengths) {
        vector<vector<int>> ret(lengths.size());
        for (int cur = 0, i = 0; i < text.length(); ++i) {
            cur = nxt[cur][text[i]];

            for (int u = cur; u; u = dict_link[u]) {
                int id = match_idx[u];
                if (id != -1) {
                    ret[id].push_back(i - lengths[id] + 1);
                }
            }
        }
        return ret;
    }
} /* init() insert() build() search() */
```

XOR Trie

```
template<typename T>
class BinaryTrie {
private:
    vector<array<int, 2>> nodes;
    vector<int> sz;
    int numberOfBits;
    T one = 1;

    bool isNodeAvailable(int cur, bool nxt) {
        return nodes[cur][nxt] != 0 && sz[nodes[cur][nxt]] > 0;
    }

    int getNode(T x) {
        int cur = 0;
        for (int i = numberOfBits - 1; i >= 0; --i) {
            bool nxt = (x >> i) & 1;
```

```
            if (nodes[cur][nxt] == 0) {
                nodes[cur][nxt] = nodes.size();
                nodes.push_back({0, 0});
                sz.push_back(0);
            }
            cur = nodes[cur][nxt];
        }
        return cur;
    }

public:
    BinaryTrie() : nodes(1, {0, 0}), sz(1, 0), numberOfBits(sizeof(T) * 8) {}

    void insert(T x, int cnt = 1) {
        int cur = 0;
        sz[cur] += cnt;
        for (int i = numberOfBits - 1; i >= 0; --i) {
            bool nxt = (x >> i) & 1;
            if (nodes[cur][nxt] == 0) {
                nodes[cur][nxt] = nodes.size();
                nodes.push_back({0, 0});
                sz.push_back(0);
            }
            cur = nodes[cur][nxt];
            sz[cur] += cnt;
        }
    }

    void erase(T x, int cnt = 1) { insert(x, -cnt); }

    T getMinXor(T x) {
        if (sz[0] == 0) throw out_of_range("The␣trie␣is␣empty");
        int cur = 0;
        T ret = 0;
        for (int i = numberOfBits - 1; i >= 0; --i) {
            bool nxt = (x >> i) & 1;
            if (isNodeAvailable(cur, nxt)) {
                cur = nodes[cur][nxt];
            } else {
                ret |= (one << i);
                cur = nodes[cur][!nxt];
            }
        }
        return ret;
    }

    T getMaxXor(T x) {
        if (sz[0] == 0) throw out_of_range("The␣trie␣is␣empty");
        int cur = 0;
        T ret = 0;
        for (int i = numberOfBits - 1; i >= 0; --i) {
            bool nxt = !((x >> i) & 1);
            if (isNodeAvailable(cur, nxt)) {
                ret |= (one << i);
                cur = nodes[cur][nxt];
            } else {
                cur = nodes[cur][!nxt];
```

```
            }
        }
        return ret;
    }

    size_t size() { return sz[0]; }
    bool contains(T x) { return sz[0] && getMinXor(x) == 0; }
    int count(T x) { return sz[getNode(x)]; }
};
```

Suffix Array

```
// O(n log(n))
struct suffix {
    int n;
    vector<int> p, c, lcp;
    string s;

    explicit suffix(string _s) : n(int(_s.size()) + 1), s(move(_s)), p(n),
        c(n), lcp(n - 1) {
        s += char(0);
        iota(p.begin(), p.end(), 0);
        sort(p.begin(), p.end(), [&](int i, int j) { return s[i] < s[j]; });
        for (int i = 1; i < n; i++) c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
        vector<int> nc(n), np(n);
        int k = 1;
        while (k < n) {
            vector<int> f(n + 1);
            for (int i = 0; i < n; i++) p[i] = (p[i] - k + n) % n, f[c[i] + 1]++;
            for (int i = 1; i <= n; i++) f[i] += f[i - 1];
            for(int i = 0; i < n; i++) np[f[c[p[i]]]++] = p[i];
            swap(p, np), nc[p[0]] = 0;
            for (int i = 1; i < n; i++)
                nc[p[i]] = nc[p[i - 1]] +
                           (c[p[i]] != c[p[i - 1]]
                            || c[(p[i] + k) % n] != c[(p[i - 1] + k) % n]);
            swap(c, nc), k <<= 1;
        }
        for(int i = k = 0; i < n - 1; i++) {
            int j = p[c[i] - 1];
            for(; s[i + k] == s[j + k]; k++);
            if(c[i]) lcp[c[i] - 1] = k;
            k = max(k - 1, 0);
        }
        s.pop_back(), c.pop_back(), n--;
        p.erase(p.begin()), lcp.erase(lcp.begin());
        for(auto &i : c) i--;
    }
    vector<vector<int>> table;
    void buildLcp() {
        int LOG = __lg(n) + 1;
        table.resize(LOG, vector<int>(n));
        table[0] = lcp;
        for(int l = 1; l < LOG; l++) {
            for(int i = 0; i + (1 << (l - 1)) < n; i++) {
```

```
                table[l][i] = min(table[l - 1][i], table[l - 1][i + (1 << (l - 1))]);
            }
        }
    }
    int query(int l, int r) { // 0-based
        if(l == r) return n - 1 - l;
        l = c[l], r = c[r];
        if(l > r) swap(l, r);
        r--;
        int len = __lg(r - l + 1);
        return min(table[len][l], table[len][r - (1 << len) + 1]);
    }
    auto count(string const &t) { // O(log(n) * size(t))
        int l = int(lower_bound(p.begin(), p.end(), -1, [&](int i, int j) {
            return s.substr(i, min<size_t>(t.size(), n - i)) < t;
        }) - p.begin());
        int r = int(lower_bound(p.begin() + l, p.end(), -1, [&](int i, int j) {
            return s.substr(i, min<size_t>(t.size(), n - i)) <= t;
        }) - p.begin()) - 1;
        return pair{l, r};
    }
};
```

Suffix Automaton

```
namespace sam {
    const int N = 4000200, SIGMA = 26; // N two times the max length
    int sa[N][SIGMA], len[N], fail[N], terminal[N], nodes, lst, C[N];

    void init() {
        nodes = lst = 0;
        memset(sa[0], -1, sizeof sa[0]);
        fail[0] = -1;
        len[0] = terminal[0] = 0;
    }

    int create_node() {
        nodes++;
        memset(sa[nodes], -1, sizeof sa[nodes]);
        terminal[nodes] = 0;
        return nodes;
    }

    void insert(char ch) {
        int u = create_node();
        len[u] = len[lst] + 1;
        int p, q;
        for (p = lst; ~p; p = fail[p]) {
            q = sa[p][ch];
            if (~sa[p][ch]) break;
            sa[p][ch] = u;
        }

        if (p == -1) {
            fail[u] = 0;
        } else if (len[p] + 1 == len[q]) {
```

```
            fail[u] = q;
        } else {
            int x = create_node();
            len[x] = len[p] + 1;
            memcpy(sa[x], sa[q], sizeof sa[0]);
            for (int v = p; ~v && sa[v][ch] == q; v = fail[v])
                sa[v][ch] = x;
            fail[x] = fail[q];
            fail[q] = fail[u] = x;
        }
        lst = u;
        C[u] = 1;
    }

    void insert(const string &s) {
        for (auto ch: s)
            insert(ch);
    }

    string walk(const string &s) {
        int cur = 0;
        for (auto ch: s) {
            if (sa[cur][ch] == -1) return "NO"s;
            cur = sa[cur][ch];
        }
        return "YES";
    }

    void pre_count() {
        vector<vector<int> > vlen(N);
        for (int i = 0; i < N; i++) vlen[len[i]].push_back(i);
        for (int l = N - 1; ~l; l--)
            for (auto x: vlen[l])
                C[fail[x]] += C[x];
    }

    int count(const string &s) {
        int cur = 0;
        for (auto ch: s) {
            if (!~sa[cur][ch]) return 0;
            cur = sa[cur][ch];
        }
        return C[cur];
    }

    vector<ll> count_distinct() {
        vector<ll> dp(nodes + 1), outdeg(nodes + 1);
        vector<vector<int> > indeg(nodes + 1);
        for (int i = 0; i <= nodes; i++)
            for (int j = 0; j < SIGMA; j++)
                if (~sa[i][j]) {
                    outdeg[i]++;
                    indeg[sa[i][j]].push_back(i);
                }
        queue<int> q;
        for (int i = 0; i <= nodes; i++)
            if (!outdeg[i])
```

```
                q.push(i);
        while (!q.empty()) {
            int top = q.front();
            q.pop();
            dp[top] = 1;
            for (int i = 0; i < SIGMA; i++)
                if (~sa[top][i])
                    dp[top] += dp[sa[top][i]];
            for (auto x: indeg[top]) {
                outdeg[x]--;
                if (!outdeg[x]) q.push(x);
            }
        }

        return dp;
    }

    ll count_distinct2() {
        ll ret = 0;
        for (int i = 1; i <= nodes; i++)
            ret += len[i] - len[fail[i]];
        return ret;
    }

    ll sum_distinct() {
        vector<ll> d(nodes + 1), ans(nodes + 1), outdeg(nodes + 1);
        vector<vector<int> > indeg(nodes + 1);
        queue<int> q;
        for (int i = 0; i <= nodes; i++) {
            for (int j = 0; j < SIGMA; j++)
                if (~sa[i][j]) {
                    outdeg[i]++;
                    indeg[sa[i][j]].push_back(i);
                }
            if (!outdeg[i]) q.push(i);
        }
        while (!q.empty()) {
            int top = q.front();
            q.pop();

            d[top] = 1;
            for (int i = 0; i < SIGMA; i++)
                if (~sa[top][i]) {
                    d[top] += d[sa[top][i]];
                    ans[top] += d[sa[top][i]] + ans[sa[top][i]];
                }

            for (int x: indeg[top])
                if (!--outdeg[x])
                    q.push(x);
        }
        return ans.front();
    }

    ll sum_distinct2() {
        ll ret = 0;
```

```
        for (int i = 1; i <= nodes; i++) {
            int a = len[i], b = len[fail[i]] + 1;
            ll num = a-b+1;
            ret += num * (a+b)/2;
        }
        return ret;
    }

    string get_kth(ll k) {
        auto d = count_distinct();
        string ret;
        int cur = 0;
        while (k) {
            for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
                if (d[sa[cur][i]] >= k) {
                    k--;
                    ret.push_back(i);
                    cur = sa[cur][i];
                    break;
                }
                k -= d[sa[cur][i]];
            }
        }
        return ret;
    }

    string smallest_shift(int n) {
        int cur = 0;
        string ret;
        while (n--) {
            for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
                cur = sa[cur][i];
                ret.push_back(i);
                break;
            }
        }
        return ret;
    }

    vector<int> vis(N);
    void dfs(int cur, const vector<ll>& d) {
        if (vis[cur]) return;
        for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
            cout << cur << "," << d[cur]-1 << '␣' << sa[cur][i] <<
                "," << d[sa[cur][i]]-1 << '␣' << char(i+'a') << '\n';
            dfs(sa[cur][i], d);
        }
    }
    void prnt() {
        auto d = count_distinct();
        dfs(0, d);
    }
} /* init() insert() */
```

Suffix Automaton Subproblems

6. Dynamic Programming

LIS

```
int lis_size(vector<int>& nums) {
    vector<int> tail;
    for (auto x : nums) {
        auto it = lower_bound(tail.begin(), tail.end(), x);
        if (it == tail.end()) tail.push_back(x);
        else *it = x;
    }
    return tail.size();
}

int lis_with_sequence(const vector<int>& a,
                      vector<int>& out_seq)
    {
        int n = a.size();
        vector<int> tail;
        vector<int> tail_idx;
        vector<int> prev(n, -1);
        tail.reserve(n);
        tail_idx.reserve(n);

        for (int i = 0; i < n; ++i) {
            int x = a[i];
            auto it = lower_bound(tail.begin(), tail.end(), x);
            int pos = it - tail.begin();
            if (it == tail.end()) {
                tail.push_back(x);
                tail_idx.push_back(i);
            }
            else {
                *it = x;
                tail_idx[pos] = i;
            }
            if (pos > 0)
                prev[i] = tail_idx[pos-1];
        }

        int lis_len = tail.size();
        out_seq.clear();
        for (int cur = tail_idx[lis_len - 1]; cur >= 0; cur = prev[cur])
            out_seq.push_back(a[cur]);
        reverse(out_seq.begin(), out_seq.end());

        return lis_len;
    }
```

0/1 Knapsack

```
int knapsack(int W, vector<int> &val, vector<int> &wt) {
    vector<int> dp(W + 1, 0);
    for (int i = 1; i <= wt.size(); i++)
        for (int j = W; j >= wt[i - 1]; j--)
            dp[j] = max(dp[j], dp[j - wt[i - 1]] + val[i - 1]);
```

```
    return dp[W];
}

int unbounded_knapSack(int capacity, vector<int> &val, vector<int> &wt) {
    vector<int> dp(capacity + 1, 0);
    for (int i = val.size() - 1; i >= 0; i--) {
        for (int j = 1; j <= capacity; j++) {
            int take = 0;
            if (j - wt[i] >= 0) {
                take = val[i] + dp[j - wt[i]];
            }
            int noTake = dp[j];
            dp[j] = max(take, noTake);
        }
    }
    return dp[capacity];
}
```

Edit Distance

```
string a, b; cin >> a >> b;
int na = a.size(), nb = b.size();
int dp[na+1][nb+1];
memset(dp, 0, sizeof dp);

for (int i = 1; i <= na; i++)
    dp[i][0] = i;
for (int j = 1; j <= nb; j++)
    dp[0][j] = j;
for (int i = 1; i <= na; i++) {
    for (int j = 1; j <= nb; j++) {
        dp[i][j] = min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]}) + 1;
        if (a[i-1] == b[j-1])
            dp[i][j] = min(dp[i][j], dp[i-1][j-1]);
    }
}
cout << dp[na][nb];
```

Longest Common Subsequence

```
void lcs(char* X, char* Y, int m, int n)
{
    int L[m + 1][n + 1];
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                L[i][j] = 0;
            else if (X[i - 1] == Y[j - 1])
                L[i][j] = L[i - 1][j - 1] + 1;
            else
                L[i][j] = max(L[i - 1][j], L[i][j - 1]);
        }
    }
    int index = L[m][n];
```

```
    char lcs[index + 1];
    lcs[index] = '\0';
    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (X[i - 1] == Y[j - 1]) {
            lcs[index - 1]
                = X[i - 1];
            i--;
            j--;
            index--;
        }
        else if (L[i - 1][j] > L[i][j - 1])
            i--;
        else
            j--;
    }
}
```

Shortest Common Supersequence

```
while (i > 0 && j > 0) {
    if (str1[i - 1] == str2[j - 1]) {
        result.push_back(str1[i - 1]);
        i--;
        j--;
    } else if (dp[i - 1][j] > dp[i][j - 1]) {
        result.push_back(str1[i - 1]);
        i--;
    } else {
        result.push_back(str2[j - 1]);
        j--;
    }
}
while (i > 0) {
    result.push_back(str1[i - 1]);
    i--;
}
while (j > 0) {
    result.push_back(str2[j - 1]);
    j--;
}
reverse(result.begin(), result.end());
return result;
```

Count Subsets Sum to K

```
int perfectSum(vector<int> &arr, int target) {
    int n = arr.size();
    vector<int> dp(target + 1, 0), ndp(target + 1, 0);
    dp[0] = 1;
    for (int i = 1; i <= n; i++) {
        ndp = dp;
        for (int j = 0; j <= target; j++) {
            if (j >= arr[i - 1]) {
                ndp[j] += dp[j - arr[i - 1]];
```

```
            }
        }
        dp = ndp;
    }
    return ndp[target];
}
```

Hamiltonian Paths

```
int rec(int u, int vis) {
    vis |= (1 << u);
    if (u == n-1)
        return __builtin_popcount(vis) == n;

    int& ans = dp[vis][u];
    if (~ans) return ans;
    // dpid[u][vis] = 2;
    ans = 0;
    each(ver, g[u])
        if (!((vis >> ver) & 1))
            ans = (ans + rec(ver, vis | (1 << ver)))%mod;
    return ans;
}
```

Kadane

```
template<class T>
array<ll, 3> kadane(const vector<T>& arr) {
    ll mx = LLONG_MIN, csum = 0;
    ll s = 0, e = 0, ts = 0, sz = arr.size();
    for (ll i = 0; i < sz; i++) {
        if (csum + arr[i] > arr[i]) {
            csum += arr[i];
        } else {
            csum = arr[i];
            ts = i;
        }
        if (csum > mx) {
            mx = csum;
            s = ts;
            e = i;
        }
    }
    return {mx, s, e};
}
```

Dynamic max subarray sum

```
struct info {
    int sum, pref, suff, ans, mnelement = -1e4-10;;
    info(int x) {
        sum = pref = suff = x;
//        ans = max<int>(x, 0);
        ans = x; // if empty subarray is not allowed
    }
```

```
    info() { // default value
        sum = pref = suff = ans = mnelement;
    }
    friend info operator+(const info &l, const info &r) {
        info ret;
        ret.sum = l.sum + r.sum;
        ret.pref = max(l.pref, l.sum + r.pref);
        ret.suff = max(r.suff, r.sum + l.suff);
        ret.ans = max({l.ans, r.ans, l.suff + r.pref});
        return ret;
    }
};
```

Meet in the middle

```
vector<int> a;
auto get_subset_sums = [&](int l, int r) -> vector<ll> {
    int len = r - l + 1;
    vector<ll> res;
    for (int i = 0; i < (1 << len); i++) {
        ll sum = 0;
        for (int j = 0; j < len; j++)
            if (i & (1 << j))
                sum += a[l + j];

        res.push_back(sum);
    }
    return res;
};
vector<ll> left = get_subset_sums(0, n / 2 - 1);
vector<ll> right = get_subset_sums(n / 2, n - 1);
sort(left.begin(), left.end());
sort(right.begin(), right.end());
ll ans = 0;
for (ll i : left) {
    auto low_iterator = lower_bound(right.begin(), right.end(), x - i);
    auto high_iterator = upper_bound(right.begin(), right.end(), x - i);
    ans += high_iterator - low_iterator;
}
```

Digit DP

```
using ull = unsigned long long int;
#define int ull
pair<int, int> dp[2][16];
bool vis[2][16];
string num;
pair<int, int> rec(int idx, bool U) {
    if (idx == num.size()) return {0, 1};
    pair<int, int>& ret = dp[U][idx];
    if (vis[U][idx]) return ret;
    vis[U][idx] = 1;
    ret = {0, 0};
    int D = U ? num[idx] - '0' : 9;
    for (int d = 0; d <= D; d++) {
        auto [x, y] = rec(idx+1, U&d==D);
```

```
        ret.first += x + d * y;
        ret.second += y;
    }
    return ret;
}
```

7. Bit Manipulation

Xor Basis

```
template<const int Log = 30, typename T = ll>
struct basis {
    int sz = 0;
    array<T, Log> a{};
    void add(T x) {
        if(sz == Log) return;
        int i;
        while(x) {
            if(!a[i = __lg(x)]) return sz++, void(a[i] = x);
            x ^= a[i];
        }
    }
    T reduce(T x) {
        if(sz == Log) return 0;
        T res = 0;
        int i;
        while(x) {
            if(a[i = __lg(x)]) x ^= a[i];
            else res |= T(1) << i, x ^= T(1) << i;
        }
        return res;
    }
    bool find(T x) {
        if(sz == Log) return true;
        int i;
        while(x) {
            if(a[i = __lg(x)]) x ^= a[i];
            else return false;
        }
        return true;
    }
    void clear() {
        if(sz) a.fill(0), sz = 0;
    }
    T getMax(T r = 0) {
        for(int i = Log - 1; i >= 0; i--) r = max(r ^ a[i], r);
        return r;
    }

    T find_k(size_t k, T base_val = 0) {
        assert(k < 1ULL << sz);
        T curr = base_val;
        for(int i = Log - 1, b = sz - 1; i >= 0; i--) {
            if(a[i]) {
                if((k >> b & 1) ^ (curr >> i & 1)) curr ^= a[i];
                b--;
            }
        }
        return curr;
    }

    T getMaxBounded(T limit, T r = 0) {
        T best = -1;
```

```
        for (int i = Log - 1; i >= 0; i--) {
            if (a[i]) {
                T r1 = r, r2 = r ^ a[i];
                if ((r1 >> i) & 1) swap(r1, r2);

                if ((limit >> i) & 1) {
                    T temp = r1;
                    for (int j = i - 1; j >= 0; j--)
                        temp = max(temp ^ a[j], temp);
                    best = max(best, temp);
                    r = r2;
                } else
                    r = r1;
            } else {
                int bit_r = (r >> i) & 1;
                int bit_l = (limit >> i) & 1;

                if (bit_l == 1 && bit_r == 0) {
                    T temp = r;
                    for (int j = i - 1; j >= 0; j--)
                        temp = max(temp ^ a[j], temp);
                    best = max(best, temp);
                    r = -1;
                    break;
                }
                if (bit_l == 0 && bit_r == 1) {
                    r = -1;
                    break;
                }
            }
        }
        if (r != -1) best = max(best, r);

        return best;
    }

    friend basis operator+(basis const &l, basis const &r) {
        if(l.sz == Log) return l;
        if(r.sz == Log) return r;
        auto res = l;
        for(int i = 0; i < Log; i++) if(r.a[i]) res.add(r.a[i]);
        return res;
    }
};
```

Max Xor Subset In Range

```
// Given queries L,R find max XOR subset in range
const int N = 5e5 + 5;
vector<pair<int,int>>qqq[N];
int ans[N],basis[22],arr[N],last[22];

void add(int ind,int x){
    for (int i = 21; i >= 0; --i) {
        if((x >> i & 1) == 0)continue;
        if(ind > last[i]){
```

```
            swap(x,basis[i]);
            swap(ind,last[i]);
        }
        x ^= basis[i];
    }
}
int query(int ind){
    int ret = 0;
    for (int i = 21; i >= 0; --i) {
        if(last[i] >= ind){
            ret = max(ret,ret ^ basis[i]);
        }
    }
    return ret;
}

void solve() {
    int n;cin >> n;
    memset(last,-1,sizeof last);
    for (int i = 0; i < n; ++i) {
        cin >> arr[i];
    }
    int q;cin >> q;
    for (int i = 0; i < q; ++i) {
        int l,r;cin >> l >> r;
        l--,r--;
        qqq[r].emplace_back(l,i);
    }
    for (int i = 0; i < n; ++i) {
        add(i,arr[i]);
        for(auto &x:qqq[i]){
            int l = x.fi;
            int ind = x.se;
            ans[ind] = query(l);
        }
    }
    for (int i = 0; i < q; ++i) {
        cout << ans[i] << endl;
    }
}
```

and (&) in range [l, r]

```
ll andRange(ll l, ll r) {
    ll ans=0, msb = -1;
    while(r>0) { r >>= 1; msb++;}
    for(ll i= msb; ~i; i--) {
        ll la = 1;
        if((l&(la << i)) == (r&(la << i)))
            ans += (l&(la << i));
        else
            break;
    }
    return ans;
}
```

Bit Twiddle Permute

```
int bit_twiddle_permute(int v) { // next integer that has _pop_count(v) bits
    int t = v | (v - 1);
    int w = (t + 1) | (((~t & -~t) - 1) >> (__builtin_ctz(v) + 1));
    return w;
}
```

8. Game Theory and Sequences

Mex Calculator

```
class mex_calculator {
    map<int, int> count;
    set<int> missing;
    public:
    mex_calculator(const vector<int>& arr, int upper_bound) { // n log n
        for (int x : arr)
            count[x]++;
        for (int i = 0; i <= upper_bound + 1; ++i)
            if (count[i] == 0)
                missing.insert(i);
    }
    void insert(int x) { // log
        count[x]++;
        if (count[x] == 1)
            missing.erase(x);
    }
    void remove(int x) { // log
        if (count[x] == 0) return;
        count[x]--;
        if (count[x] == 0)
            missing.insert(x);
    }
    int get_mex() { // 1
        return *missing.begin();
    }
};
```

Remove Game

```
void solve() {
    int n; cin >> n;
    vector<int> v(n);
    getv(v);
    ll diff[n+1][n+1]{}; // mx diff p1-p2 on interval [l, r]
    for (int l = n-1; ~l; l--) {
        for (int r = 0; r < n; r++) {
            // dp[l][r] = mx( v[l]-dp[l+1][r], v[r]-dp[l][r-1] )
            if (l == r)
                diff[l][r] = v[l];
            else diff[l][r] = max<ll>(v[l] - diff[l+1][r], v[r]-diff[l][r-1]);
        }
    }

    cout << (accumulate(all(v), 0ll) + diff[0][n-1])/2;
}
```

K-th Balanced Bracket Sequence

```
//O(n^2)
string kth_balanced(int n, int k) {
    vector<vector<int>> d(2*n+1, vector<int>(n+1, 0));
```

```cpp
    d[0][0] = 1;
    for (int i = 1; i <= 2*n; i++) {
        d[i][0] = d[i-1][1];
        for (int j = 1; j < n; j++)
            d[i][j] = d[i-1][j-1] + d[i-1][j+1];
        d[i][n] = d[i-1][n-1];
    }
    string ans;
    int depth = 0;
    for (int i = 0; i < 2*n; i++) {
        if (depth + 1 <= n && d[2*n-i-1][depth+1] >= k)
        {
            ans += '(';
            depth++;
        } else {
            ans += ')';
            if (depth + 1 <= n)
                k -= d[2*n-i-1][depth+1];
            depth--;
        }
    }
    return ans;
}
```

Next Balanced Sequence

```cpp
bool next_balanced_sequence(string & s) { // O(n)
    int n = s.size();
    int depth = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (s[i] == '(')
            depth--;
        else
            depth++;
        if (s[i] == '(' && depth > 0) {
            depth--;
            int open = (n - i - 1 - depth) / 2;
            int close = n - i - 1 - open;
            string next = s.substr(0, i) + ')' +
            string(open, '(') + string(close, ')');
            s.swap(next);
            return true;
        }
    }
    return false;
}
```

9. Geometry

Geometry Notes

```
Generate 2 points on a line
// ax + by + c = 0
if(a == 0){
        p1 ={0,-1.0 * c/b};
        p2 = {1,-1.0 * c/b};
    }
    else{
        p1 = {-1.0* c/a,0};
        p2 = {-1.0 * (c + b)/a,1};
    }'


Youre given 4 integers which are the
coefficients A B and C of the normal equation of the
straight line and a distance value R.

void solve() {
    double a,b,c,r;cin >> a >> b >> c >> r;
    double base = sqrt(a * a + b * b);
    a /= base;
    b /= base;
    c /= base;
    cout << setprecision(15) << a << '␣' << b << '␣' << c + r << endl;
    cout << setprecision(15) << a << '␣' << b << '␣' << c - r << endl;
}

Area of the sector without an angle = (l * r) / 2
The length of the arc l = (theta / 360) * 2 * pi * r

The area of the parallelogram = the cross-product of
    2 adjacent sides = 2 * area of the triangle made by 3 points.

Given 1 side of the Pythagorean Triangle ... Get the missing 2 sides :
ll n;
cin >> n;
if (n == 1 || n==2)
    cout << -1;
else if (n & 1)
    cout << (n * n + 1) / 2 << "␣" << (n * n - 1) / 2;
else
    cout << n * n / 4 + 1 << "␣" << n * n / 4 - 1;

In triangle abc angle bac cos(theta) = (b^2 + c^2 - a^2) / (2 * b * c)

n = number of sides of a regular polygon
S = side length of the polygon
ap = apothem the distance from the center of the polygon to the middle of any side
r = radius of the polygon which is the distance from the center of the polygon to any corner.
p = perimeter of the polygon

p = S * n
ap = S / (2 * tan(180/n)) = r * cos(180/n)
r = S / (2 * sin(180/n)) = ap / cos(180/n)
Area = (p * ap)/2 , (S^2 * n) / (4 * tan(180/n)) = ap^2 * n * tan(180/n)
```

```
    = (r^2 * n * sin(360/n))/2

sin(2*theta) = 2 * sin(theta) * cos(theta)
cos(2*theta) = cos(theta)^2 - sin(theta)^2 = 2 * cos(theta)^2 - 1
    = 1 - 2 * sin(2*theta)^2
sin(theta)^2 = (1 - cos(2 * theta))/2
cos(theta)^2 = (1 + cos(2 * theta))/2
tan(2*theta) = (2 * tan(theta)) / (1 - tan(theta)^2)

Circle intersection r1,r2,d where r1 >= r2
If d = r1 + r2 they touch from outside
If d = r1 - r2 they touch from inside
If r1 - r2 < d < r1 + r2 they intersect in two points

Plane equation ax + by + cz + d = 0
AB = (Bx-Ax,By-Ay,Bz-Az)
AC = (Cx-Ax,Cy-Ay,Cz-Az)
AB x AC = (a,b,c)
a = (By-Ay)*(Cz-Az)-(Cy-Ay)*(Bz-Az)
b = (Bz-Az)*(Cx-Ax)-(Cz-Az)*(Bx-Ax)
c = (Bx-Ax)*(Cy-Ay)-(Cx-Ax)*(By-Ay)
d = -(a*Ax+b*Ay+c*Az)

// Checks if four points lie in the same plane or not
bool samePlane(point a,point b,point c){
    // a * (b x c) = volume = 0
    return (a.dot(b.cross(c)) == 0);
}

void solve() {
    vector<point>v(4);
    for (int i = 0; i < 4; ++i) {
        cin >> v[i].x >> v[i].y >> v[i].z;
    }
    for (int i = 0; i < 4; ++i) {
        v[i].x -= v[3].x;
        v[i].y -= v[3].y;
        v[i].z -= v[3].z;
    }
    cout << (samePlane(v[0],v[1],v[2]) ? "YES":"NO") << endl;
}
```

Geometry (X, Y, dot, cross)

```
/*
 conj(a) -> a.imag() *= -1
 abs(point) distance between (0,0) to this point
 norm(point) squared magnitude -> real² + imag²
 hypot(x, y) -> sqrt(x² + y²)
 arg(vector) angle between this vector and x-axis
 clamp(a, l, r) == min(r, max(l, a))
 polar(rho, theta) -> make vector with length rho and angle theta
 internal angle = (n - 2) * 180 / n
 number of diagonals n * (n - 3) / 2
 Area(p) = internal_points_cnt + (boundary_points/2) - 1
 boundary_point in vector = gcd(|x2-x1|, |y2-y1|) + 1
```

```
 line have infinity point, segment have to end points
 vector(x, y) perpendicular to vector(-y, x) and (y, -x)
*/

using ll = int64_t;

using ld = double;
using pt = complex<ld>;

const ll INF = 7e18;
const ld EPS = 1e-9;
const ld PI = acos(-1);

#define X real()
#define Y imag()

#define dot(a, b) (conj(a) * (b)).X
#define cross(a, b) (conj(a) * (b)).Y

int sign(ld x) {
    return (x > EPS) - (x < -EPS);
}

struct compX{
    bool operator()(pt a, pt b) const {
        return a.X != b.X ? a.X < b.X : a.Y < b.Y;
    }
};
struct compY{
    bool operator()(pt a, pt b) const {
        return a.Y != b.Y ? a.Y < b.Y : a.X < b.X;
    }
};

// ================ line, segment ==========================

// projection of pt p onto line ab
pt project(pt a, pt b, pt p) {
    pt ab = b - a;
    return a + ab * dot(p - a, ab) / norm(ab);
}

// works for any orientation
bool onSegment(pt a, pt b, pt p) {
    return sign(cross(b - a, p - a)) == 0 &&
           sign(dot(p - a, p - b)) <= 0;
}

// ccw: >0 left, <0 right, =0 collinear
int ccw(pt a, pt b, pt c) {
    return sign(cross(b - a, c - a));
}

// works for any pts
ld distanceToLine(pt a, pt b, pt p) {
    return fabsl(cross(b - a, p - a)) / abs(b - a);
}
```

```
// works for any line
ld distanceToLine(ld A, ld B, ld C, pt p) {
    return fabsl(A*p.X + B*p.Y + C) / abs(pt(A, B));
}

// works for any pts
ld distanceToSegment(pt a, pt b, pt p) {
    if (dot(b - a, p - a) < 0) return abs(p - a);
    if (dot(a - b, p - b) < 0) return abs(p - b);
    return distanceToLine(a, b, p);
}

// works for intersecting lines (not parallel)
pt lineIntersect(pt a, pt b, pt c, pt d) {
    pt ab = b - a, cd = d - c;
    return a + ab * (cross(c - a, cd) / cross(ab, cd));
}

// works for all segments (returns intersection pt if exists)
bool segmentsIntersect(pt a, pt b, pt c, pt d, pt &inter) {
    int d1 = ccw(a, b, c), d2 = ccw(a, b, d);
    int d3 = ccw(c, d, a), d4 = ccw(c, d, b);

    if(d1 * d2 < 0 && d3 * d4 < 0)
        return inter = lineIntersect(a, b, c, d), true;

    if(d1 == 0 && onSegment(a, b, c)) return inter = c, true;
    if(d2 == 0 && onSegment(a, b, d)) return inter = d, true;
    if(d3 == 0 && onSegment(c, d, a)) return inter = a, true;
    if(d4 == 0 && onSegment(c, d, b)) return inter = b, true;

    return false;
}

// works for any triangle
ld triangleArea(pt a, pt b, pt c) {
    return 0.5 * fabs(cross(b - a, c - a));
}

bool ptInTriangle(pt a, pt b, pt c, pt p) {
    ld s1 = cross(b - a, p - a);
    ld s2 = cross(c - b, p - b);
    ld s3 = cross(a - c, p - c);
    return (sign(s1) >= 0 && sign(s2) >= 0 && sign(s3) >= 0) ||
           (sign(s1) <= 0 && sign(s2) <= 0 && sign(s3) <= 0);
}

// angle abc in radians
ld angle_abc(pt a, pt b, pt c) {
    return acos(clamp<ld>(dot(a - b, c - b) / (abs(a - b) * abs(c - b)), -1, 1));
}

// ============================ Circles ================================

pair<ld, pt> findCircle(pt a, pt b, pt c) {
```

```
    pt m1 = (a + b) / 2.0, m2 = (b + c) / 2.0;
    pt ab = b - a, bc = c - b;
    pt center = lineIntersect(m1, m1 + pt(-ab.Y, ab.X),
                              m2, m2 + pt(-bc.Y, bc.X));
    return {abs(center - a), center};
}

vector<pt> lineCircleIntersect(pt a, pt b, pt center, ld r) {
    pt ab = b - a, ao = center - a;
    pt proj = a + ab * dot(ao, ab) / norm(ab);
    ld d = abs(proj - center);
    if (d > r + EPS) return {};
    if (abs(d - r) < EPS) return {proj};
    ld h = (ld)sqrtl(r*r - d*d);
    pt dir = ab / abs(ab);
    return {proj + dir * h, proj - dir * h};
}

// in 0, 1, 2 pts
vector<pt> circleCircleIntersect(pt c1, ld r1, pt c2, ld r2) {
    ld d = abs(c2 - c1);
    if(d > r1 + r2 + EPS || d < abs(r1 - r2) - EPS) return {};
    if(abs(d) < EPS && abs(r1 - r2) < EPS) return vector(3, c1); // infinity intersection

    ld a = (r1*r1 - r2*r2 + d*d) / (2 * d), h2 = r1*r1 - a*a;
    if (h2 < -EPS) return {};

    pt dir = (c2 - c1) / d, p = c1 + dir * a;
    if (abs(h2) < EPS) return {p};
    ld h = sqrt(h2);
    pt offset = dir * pt(0, 1) * h;
    return {p + offset, p - offset};
}

pair<ld, pt> minimumEnclosingCircle(vector<pt> p) {
    using circle = pair<ld, pt>;
    shuffle(p.begin(), p.end(), mt19937(random_device{}()));
    auto contains = [](circle c, const vector<pt>& pts) {
        return all_of(pts.begin(), pts.end(),
                      [&](auto p) {return abs(p - c.second) <= c.first + EPS;});
    };
    auto circleFrom2 = [](pt a, pt b) {
        pt c = (a + b) / 2.0;
        return circle{abs(a - c), c};
    };
    auto circleFrom3 = [](pt a, pt b, pt c) {
        pt ab = (a + b) / 2.0, ac = (a + c) / 2.0;
        pt ab_perp = (b - a) * pt(0, 1), ac_perp = (c - a) * pt(0, 1);
        pt o = lineIntersect(ab, ab + ab_perp, ac, ac + ac_perp);
        return circle{abs(o - a), o};
    };
    vector<pt> R;
    function<circle(int)> welzl = [&](int n) -> circle {
        if (n == 0 || R.size() == 3) {
            if (R.empty()) return {};
            if (R.size() == 1) return {0, R[0]};
```

```
            if (R.size() == 2) return circleFrom2(R[0], R[1]);
            return circleFrom3(R[0], R[1], R[2]);
        }
        pt q = p[n - 1];
        circle D = welzl(n - 1);
        if (contains(D, {q})) return D;
        R.push_back(q);
        auto res = welzl(n - 1);
        R.pop_back();
        return res;
    };
    return welzl((int)p.size());
}

// ==================== polygon ===========================

// works for any polygon (returns +1 for ccw, -1 for cw)
ld polygonSign(vector<pt>& p) {
    ld area = 0;
    int n = (int)p.size();
    p.push_back(p[0]);
    for(int i = 0; i < n; ++i) area += cross(p[i], p[i + 1]);
    p.pop_back();
    return sign(0.5 * area);
}

// works for any polygon (removes dups, enforces ccw order)
void normPolygon(vector<pt>& p) {
    vector<pt> res;
    for(auto i : p) if(res.empty() || abs(i - res.back()) > EPS)
            res.push_back(i);

    if(res.size() > 1 && abs(res.front() - res.back()) < EPS)
        res.pop_back();

    if(polygonSign(res) < 0) reverse(res.begin(), res.end());

    p = res;
}

// works for simple polygons with integer coordinates
ll internalPointsCount(vector<pt>& p) {
    ll A2 = 0, B = 0;
    int n = (int)p.size();
    p.push_back(p[0]);
    for (int i = 0; i < n; ++i) {
        pt a = p[i], b = p[i + 1];
        A2 += ll(a.X * b.Y - a.Y * b.X);
        B += __gcd((ll)abs(b.X - a.X), (ll)abs(b.Y - a.Y));
    }
    p.pop_back();
    return (abs(A2) - B + 2) / 2;
}

// works for any polygon (cw or ccw, convex or not)
ld polygonArea(const vector<pt>& p) {
```

```
    int n = (int)p.size();
    ld area = 0;
    for (int i = 0; i+1 < n; ++i)
        area += cross(p[i], p[i + 1]);
    area += cross(p.back(), p.front());
    return fabsl(area) / 2.0;
}

// works for any polygon (cw or ccw, convex or not)
bool ptInPolygon(const vector<pt> &p, pt o) {
    int in = 0, n = (int)p.size();
    for (int i = 0; i+1 < n; ++i) {
        pt a = p[i], b = p[i + 1];
        if (onSegment(a, b, o)) return true;
        if (a.Y > o.Y != b.Y > o.Y) {
            ld x = a.X + (b.X - a.X) *
                         (o.Y - a.Y) / (b.Y - a.Y);
            if(x > o.X) in ^= 1;
        }
    }
    {
        pt a = p.back(), b = p.front();
        if (onSegment(a, b, o)) return true;
        if ((a.Y > o.Y) != (b.Y > o.Y)) {
            ld x = a.X + (b.X - a.X) *
                         (o.Y - a.Y) / (b.Y - a.Y);
            if(x > o.X) in ^= 1;
        }
    }
    return in;
}


// work for simple convex polygon
bool ptInConvex(vector<pt> &poly, pt p) {
    int n = int(poly.size());
    if(n == 1) return sign(abs(poly[0] - p)) == 0;
    if(n == 2) return onSegment(poly[0], poly[1], p);

    pt f = poly[0];

    if(sign(cross(poly[1] - f, p - f)) < 0 ||
        sign(cross(poly[n - 1] - f, p - f)) > 0) return false;

    int l = 1, r = n - 1;
    while(r > l + 1) {
        int mid = (l + r) >> 1;
        if(sign(cross(poly[mid] - f, p - f)) > 0) l = mid;
        else r = mid;
    }
    return ptInTriangle(f, poly[l], poly[r], p);
}

// works for any simple polygon (cw or ccw)
pt polygonCentroid(const vector<pt>& p) {
    ld A = 0, c;
```

```
    pt C(0, 0);
    int n = (int)p.size();
    pt cur, nxt;
    for (int i = 0; i+1 < n; ++i) {
        cur = p[i], nxt = p[i + 1];
        c = cross(cur, nxt);
        A += c;
        C += (cur + nxt) * c;
    }
    cur = p.back(), nxt = p.front();
    c = cross(cur, nxt);
    A += c;
    C += (cur+nxt) * c;

    A *= 0.5;
    if (abs(A) < EPS) return C;
    return C / (6.0 * A);
}
```